ASPIRE TECH · CRM MIGRATION PROGRAMME

Replacing GoHighLevel with Twenty + n8n

A complete, feature-by-feature replacement guide.
Every GoHighLevel capability, what replaces it, and how to build it.

Prepared for: Aspire Tech / Aspire TSS — CRM Migration, Phase 1

Scope: Full GoHighLevel decommission. Twenty (self-hosted) as system of record, n8n as orchestration.

Covers: 14 workflow action categories · 35+ workflow triggers · 23 feature areas · 111 catalogued features

Status: Design reference — precedes the usage audit, does not replace it

Date: 11 August 2026 · **Revision:** 1.0

Contents

Part I · Foundations

1. Executive summary

Aspire Tech is decommissioning GoHighLevel and moving to a self-hosted Twenty CRM with n8n as the automation layer. This document specifies how every GoHighLevel capability is replaced, in enough detail to build from.

The central finding is simple and needs stating before anything else:

> **THE ONE CONCLUSION THAT SHAPES EVERYTHING**
>
> **Twenty and n8n together replace GoHighLevel's system-of-record and automation layers completely. They do not replace its communication infrastructure at all.** Twenty cannot send bulk email, SMS or voice; it cannot host forms, landing pages or booking pages. n8n orchestrates sending but is not itself a sender. Roughly one third of GoHighLevel's surface therefore requires a third-party service — not because of a configuration gap, but because those capabilities do not exist in either tool at any setting.

This is not a reason to abandon the migration. It is a reason to budget for it correctly. GoHighLevel bundles a CRM, a marketing automation platform, an email service provider, a telephony carrier reseller, a website builder, a scheduling tool, a course platform and a payments front-end into one subscription. Unbundling that stack means replacing each part explicitly.

What the replacement looks like

Layer	Replaced by	Coverage
System of record Contacts, companies, deals, subscriptions, custom objects	Twenty (self-hosted)	Complete. Twenty's custom object model is stronger than GoHighLevel's — this part of the migration is an upgrade, not a compromise.
Automation and orchestration Workflows, triggers, conditions, routing	Twenty workflows + n8n	Complete for logic. Every GoHighLevel trigger and action has a mapping in §12–13. Simple record automations stay in Twenty; anything multi-step, external or conditional moves to n8n, which is considerably more capable than GoHighLevel's builder.
Communication execution Email, SMS, voice, chat	Third-party vendors, driven by n8n	**Not covered by Twenty or n8n.** Requires an email service provider and a communications platform. See §5.
Web presence and capture Forms, funnels, sites, booking	Third-party vendors, integrated via n8n	**Not covered.** Requires a form tool, a scheduling tool, and a CMS — unless already owned elsewhere, which is likely for at least the website.
Commerce Invoices, payments, quotes	Finance system + Twenty records	Records live in Twenty; billing execution stays with the finance stack.

The numbers

Against a 111-feature catalogue covering 23 GoHighLevel areas:

Disposition	Count	Meaning
TWENTY	15	Native Twenty capability. Configuration only, no build.
N8N	13	n8n reproduces it without Twenty involvement.
TWENTY+N8N	20	Record in Twenty, logic in n8n.
VENDOR	52	Needs a third-party service. Consolidates into **eight vendor decisions**, of which four plausibly disappear once usage evidence arrives.
DROP	11	Not in use, or duplicated elsewhere in Aspire's stack.

> **READ THIS NUMBER CORRECTLY**
>
> 52 vendor-dependent features is an **upper bound taken before the usage audit**. It counts every GoHighLevel capability that exists, not every capability Aspire uses. Four of the eight vendor clusters — web presence, commerce, memberships, reputation — are likely to collapse substantially once measured, because Aspire already runs a website, a finance system and a dedicated e-learning platform outside GoHighLevel. The realistic landing point is **three to five genuine vendor decisions**. That must be demonstrated by evidence, not asserted here.

What is genuinely better after the migration

- **Automation capability increases.** n8n's branching, error handling, sub-workflows, code execution and retry semantics exceed GoHighLevel's workflow builder by a wide margin. Automations that were awkward or impossible in GoHighLevel become routine.
- **The data model fits the business.** GoHighLevel forces recurring services into a deal-shaped hole. Twenty's custom objects model subscriptions, compliance engagements, training seats and phishing baselines as first-class records.
- **Automation becomes observable.** GoHighLevel does not expose workflow execution history through its API — a fact that materially obstructed this very audit. n8n exposes everything, and writing runs back into Twenty makes automation health a CRM report.
- **No per-seat pricing on the CRM.** Self-hosted Twenty removes licence cost, though not operational cost.

What gets harder

- **More systems to operate.** One vendor becomes four to six. Each has its own credentials, failure modes, billing and support relationship.
- **Deliverability becomes Aspire's problem.** GoHighLevel managed sending reputation invisibly. That responsibility now sits with a named owner.
- **The unified inbox does not survive intact.** This is the single most-felt day-to-day loss for sales users. §17 addresses it directly rather than pretending otherwise.
- **Nothing is bundled.** Every integration between layers is a workflow someone maintains.

2. How to read this document

Each feature section follows the same structure, so a reader can go straight to the capability they care about:

Heading	Contains
What GoHighLevel does	The capability as it exists today, in operational terms.
Replacement	The disposition tag and the specific components involved.
How to build it	Step-by-step construction — Twenty configuration, n8n node sequence, API calls.
Gotchas	What breaks, what is lost, what to verify before trusting it.

Disposition tags

`TWENTY` Native Twenty. `N8N` n8n alone. `TWENTY+N8N` Record in Twenty, logic in n8n. `VENDOR` Third-party service required. `DROP` Retire.

`VENDOR` always names the category of service and what it must do, never a single product. Product selection requires pricing against Aspire's real volumes, and those volumes come from the usage audit. Recommending a specific tool before the volume data exists would be a guess, and the guess would be wrong in whichever direction costs most.

Relationship to the usage audit

This document is the **design reference**: it describes how each capability is replaced, assuming Aspire uses it. The companion usage audit determines which capabilities Aspire actually uses. Both are needed, and in this order of application:

1. The audit produces evidence of what is in use.
2. Features with no usage evidence are dropped — no build required.
3. This document supplies the build specification for everything that survives.

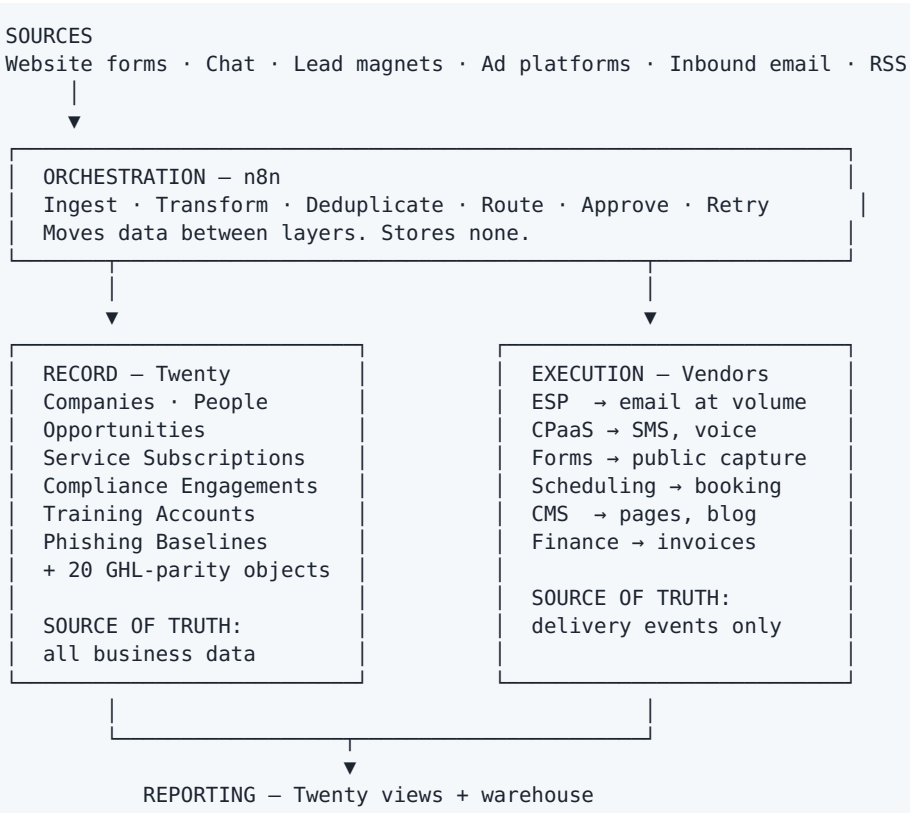
> **DO NOT BUILD FROM THIS DOCUMENT ALONE**
>
> Building every section here would replace capabilities nobody uses. The audit typically removes a third to a half of the surface. Use the audit to decide what to build; use this document to decide how.

Verification status of claims

Statements about GoHighLevel's API and Twenty's capabilities were verified against primary documentation during preparation. Where a claim could not be verified, the text says so. The complete GoHighLevel trigger and action inventories in §12–13 come from HighLevel's own support documentation, not from recollection. Sources are listed in Appendix D.

3. The replacement architecture

Three layers, plus AI as a cross-cutting concern. The layering exists to enforce one rule.



THE RULE THAT GOVERNS EVERYTHING

Each field has exactly one source of truth. Twenty owns business data. Vendors own delivery events. n8n owns nothing — it only moves data. Every sync rule, migration decision and consent design in this document follows from that single constraint. Do not let a later "simplification" put the same field in two places.

Why n8n stores nothing

It is tempting to let n8n keep state — a lookup table of external IDs, a list of who has been contacted, a counter. Resist it. State in an orchestrator is invisible to the business, unbacked up, and lost when a workflow is rewritten. Anything worth remembering belongs in Twenty as a record, where it is queryable, auditable and included in backups.

The one permitted exception is the external-ID mapping table used for idempotent upserts during migration (§14), and even that is better held as a field on the Twenty record itself.

Direction of flow

Flow	Mechanism	Notes
Twenty → n8n	Webhook, HMAC-signed	Twenty pushes on record create/update/delete. n8n must validate the signature before processing.
n8n → Twenty	REST API, Bearer key	Twenty has no inbound webhook handler. All writes go through the Core API.
Vendor → n8n	Vendor webhooks	Delivery, bounce, unsubscribe, reply, booking, payment events.
n8n → Vendor	HTTP Request or dedicated node	Send, schedule, provision.
Twenty internal	Twenty workflows	Simple field updates and record creation that never leave Twenty.

Two constraints that shape every integration

1. **Twenty does not filter webhook events at source.** All event types are delivered to a subscribed endpoint. n8n must filter early and exit cheaply on events it does not care about, or every workflow wakes for every change in the workspace.
2. **Twenty has no inbound webhook receiver on objects.** Twenty sends webhooks; it does not receive them. Anything writing into Twenty goes through n8n calling the Core API. There is no way for an external system to push directly into a Twenty record.

4. The capability boundary — what Twenty and n8n cannot do

This section is the load-bearing analysis of the document. Everything classified **VENDOR** later traces back to a line in these tables.

4.1 Twenty — verified capabilities

Twenty's automation surface, confirmed against its own documentation:

Workflow triggers	Workflow actions
Record is created Record is updated (optionally field-scoped) Record is created or updated Record is deleted Manual (Cmd+K or navbar; global, single or bulk) On a schedule (minutes/hours/days or cron, UTC) Webhook (inbound GET or POST)	Create Record · Update Record · Delete Record Search Records (max 200 results) Upsert Record Iterator (loop an array from a prior step) Filter · Delay Send Email (synced mailbox) Form (collect input mid-workflow; manual triggers only) Code (JavaScript on server-side logic functions) HTTP Request AI Agent (marked coming soon)

This is a competent automation engine for record-centric work. It is not a marketing automation platform, and the difference is concentrated in one action.

THE SINGLE MOST CONSEQUENTIAL LIMITATION

Twenty's `Send Email` action sends from a **synced mailbox**, and the documentation states plainly that **only one recipient is possible at the moment**. There is no template library, no merge-field engine, no list segmentation at send time, no suppression list, no bounce handling, no open or click tracking, no sending-domain management and no warm-up. It is a transactional send from a person's mailbox, not a campaign engine.

Every GoHighLevel feature that reaches an audience through a channel inherits this limitation. That single sentence produces most of the vendor requirements in §5.

4.2 Capabilities absent from Twenty at any configuration

Capability	Why it cannot be configured in	Consequence
Bulk / campaign email	No campaign engine; send action is one-recipient, mailbox-bound	ESP required
Deliverability infrastructure	No dedicated IP, sending domain management, DKIM/SPF/DMARC tooling, warm-up scheduling or reputation monitoring	ESP required; 2–6 week lead time
Suppression / unsubscribe enforcement	No suppression list, and nothing at send time to consult one	Consent enforcement moves to n8n (§21)
SMS / MMS	No SMS capability of any kind	CPaaS required
Voice, IVR, call recording, phone numbers	No telephony	CPaaS required; number porting and A2P registration have multi-week lead times
Public forms	The `Form` action collects input from a logged-in user during a manual workflow run; it is not a public web form	Form vendor required
Landing pages, funnels, websites	No page builder or hosting	CMS required — verify what Aspire already owns
Booking pages / availability	No scheduling engine, no availability calculation, no public booking surface	Scheduling vendor required
Invoicing, payments, CPQ	No billing engine, no document generation	Finance system + e-sign vendor
Memberships, courses, client portal	No LMS or portal	Likely already served by aspireelearning.com
Unified omnichannel inbox	Email threads sync to records; no SMS, chat or social inbox	See §17 — architectural decision required
Native mobile application	Web UI is responsive but not field-optimised	Confirm whether anyone uses the GoHighLevel mobile app before treating this as a gap

4.3 n8n — what it does and does not add

n8n closes the logic gap comprehensively. It does not close the channel gap at all.

n8n covers	n8n does not cover
Multi-step branching beyond Twenty's Filter	Being an email sender (it calls one)
Loops, batching, rate-limited iteration	Being an SMS or voice carrier
Arbitrary code (JavaScript, Python)	Hosting a public page or form
Scheduled and cron work	Managing sending reputation
Inbound webhooks from any source	Storing business data
Retry, error workflows, dead-letter patterns	Providing an end-user interface
Sub-workflows and reusable components	Regulatory registration (A2P, sender verification)
LLM chains and AI agents	
Any HTTP API on earth	
Human approval gates via Wait + resume URL	

WHERE THE MIGRATION GAINS

On automation logic specifically, the move is an **upgrade**. n8n's error handling alone — per-node retry with backoff, continue-on-error branches, and a workspace-wide Error Trigger workflow — has no equivalent in GoHighLevel, where a failed step fails silently and is discovered weeks later. Automations that were fragile in GoHighLevel become observable and recoverable.

5. The vendor layer

The 52 vendor-dependent features consolidate into eight decisions. Presented as 52 problems this looks like a failed plan; presented as eight decisions, four of which may evaporate, it is a manageable procurement exercise.

#	Cluster	What the service must do	Selection criteria	Survives audit?
A	**Email service provider**	Transactional + bulk send, templates with merge fields, suppression list, DKIM/SPF/DMARC, dedicated sending domain, bounce/complaint webhooks, open/click tracking, API + webhooks	Monthly volume; suppression import; webhook granularity; warm-up support	**Certain.** Aspire runs a newsletter and outbound sequences
B	**Communications platform (CPaaS)**	Phone numbers, SMS/MMS, voice, IVR, recording, A2P 10DLC registration, inbound webhooks	Number porting support; A2P handling; per-message cost at volume	**Verify.** Depends whether sales actually dials from GoHighLevel
C	**Web presence**	Landing pages, funnels, blog, hosting, form embedding, analytics	Whether aspiretss.com is already hosted elsewhere	**Likely shrinks sharply**
D	**Scheduling**	Public booking pages, availability from connected calendars, round-robin, reminders, reschedule/cancel links, API + webhooks	Round-robin support; webhook completeness; calendar integration	**Likely yes.** Demo booking is core to the sales motion
E	**Forms**	Public hosted forms, conditional logic, file upload, spam protection, webhook delivery	Conditional logic depth; webhook reliability; embedding	**Likely yes**

F	**Commerce and documents**	Invoicing, payment collection, subscription billing, quote/ proposal documents, e-signature	Existing finance system boundary	**Verify.** Finance may already invoice outside GoHighLevel
G	**Learning / portal**	Course delivery, enrolment, certificates, client portal	Overlap with aspireelearning.com	**Probably drops**
H	**Reputation and social**	Review request, review monitoring, social scheduling	Whether marketing already owns tooling	**Verify**

5.1 Selection sequence

Not all eight are equally urgent. Two are on the critical path and must be decided first, because their lead times run in parallel with nothing else.

Order	Cluster	Why this order	Blocks
1	A — Email service provider	Domain warm-up takes 2–6 weeks and cannot start until the vendor exists	All email features; the entire cutover date
2	B — CPaaS (if SMS/voice is in use)	A2P 10DLC registration is carrier-side regulatory review, measured in weeks	All SMS features; number porting
3	D — Scheduling	Short lead time, high daily visibility	Booking, reminders
4	E — Forms	Short lead time	Lead capture
5	C, F, G, H	Confirm scope from audit before selecting anything	—

> **VENDOR SELECTION IS THE CRITICAL PATH, NOT THE MIGRATION**
>
> Every engineering task in this document can be done in parallel with, or after, vendor selection. Neither email warm-up nor A2P registration can. If the cutover date is set from the engineering estimate alone it will be wrong by roughly a month, in the direction that causes campaigns to land in spam and SMS to be filtered by carriers.
>
> **Decide clusters A and B before starting the build.**

5.2 What "integrated via n8n" means concretely

Every vendor is integrated the same way, which keeps the architecture uniform regardless of which products are chosen:

1. **Outbound:** n8n calls the vendor API — a dedicated node where one exists, otherwise HTTP Request with a stored credential.
2. **Inbound:** the vendor posts events to an n8n Webhook node with a signed or token-protected URL.
3. **Write-back:** n8n writes the outcome into Twenty via the Core API — a `messageLog`, `appointment`, `payment` or `consentRecord` record.

4. **Failure:** the node retries; persistent failure routes to the error workflow and a dead-letter record.

Because the pattern is identical for every vendor, swapping one later touches one sub-workflow rather than the whole estate. Build the vendor call as a **sub-workflow with a stable input contract** from the start — this is the single highest-value structural decision in the build (§14.4).

6. Prerequisites and environment

6.1 Twenty

Item	Requirement
Deployment	Self-hosted, Docker. Postgres + Redis + server + worker, all four healthy — the UI can look correct while the worker is dead, so verify the worker separately.
Resources	4 GB RAM minimum. Postgres needs `shm_size: 256m` and tuned `shared_buffers` / `work_mem` before the workspace passes roughly 10,000 records.
`ENCRYPTION_KEY`	Generate with `openssl rand -base64 32`, store in a secrets manager. **Losing it loses every stored secret including OAuth tokens.** Set a dedicated key before upgrading to v2.5, where secrets move to a versioned `enc:v2:` envelope.
Postgres password	Set deliberately at first run. The value is baked into the volume; changing it later without removing the volume breaks authentication.
`LOGIC_FUNCTION_TYPE`	`LOCAL` or `LAMBDA`, or the workflow `Code` action reports "logic function execution is disabled".
`FRONT_BASE_URL` / `SERVER_URL`	Must match the access domain including scheme, or login enters a redirect loop.
API key	Settings → API & Webhooks → Create key. Scope it via Settings → Members → Roles → Assignment.
Reverse proxy	TLS termination, plus stream-response configuration or the AI chat will not respond until refresh.

6.2 Twenty API — the operational limits that shape workflow design

Property	Value
Core API	`/rest/` and `/graphql/` — CRUD on all objects, standard and custom
Metadata API	`/rest/metadata/` and `/metadata/` — objects, fields, relations
Authentication	`Authorization: Bearer <API_KEY>`
Rate limit	**100 requests per minute**
Batch size	**60 records per call.** GraphQL additionally supports batch upsert
Schema behaviour	Per-workspace. A new custom object immediately gets REST and GraphQL endpoints under its own name
Playground	Settings → API & Webhooks, after a key exists

> **100 REQUESTS PER MINUTE IS THE NUMBER THAT WILL BITE**
>
> It is far tighter than GoHighLevel's 100 per 10 seconds. Any n8n workflow that loops record-by-record against Twenty will exhaust it. **Use batch operations of up to 60 records** for migration and nightly jobs, and add `Loop Over Items` with a batch size and interval for anything iterative. Design for this from the first workflow rather than discovering it during migration load.

6.3 n8n

Item	Requirement
Deployment	Self-hosted alongside Twenty. Queue mode if concurrent executions become significant.
Credentials	One credential per vendor, stored in n8n's credential store — never inline in a node, never in workflow JSON.
Twenty integration	**HTTP Request node against the REST API.** Do not use a community Twenty node: available ones have very low adoption and single maintainers, and the entire automation estate would depend on one.
Error workflow	One workspace-wide workflow beginning with the `Error Trigger` node, set as the error workflow on every production workflow. Non-negotiable — see §14.2.
Environments	Separate n8n instances or at minimum separate credential sets for build and production, so a test run cannot send to a real contact.
Backups	Workflow JSON exported to Git. n8n's own database is not a sufficient backup of business logic.

6.4 Naming conventions — adopt before the first workflow

Two hundred workflows named after what someone was thinking that afternoon is the most common way an n8n estate becomes unmaintainable. Fix it at workflow one.

```
Workflows    [AREA] Verb Object — Trigger
             [LEAD] Create Person — Web Form
             [SUB]  Notify Renewal 90d — Schedule
             [SYS]  Error Handler — Error Trigger
             [VEND] Send Email — Sub-workflow

Areas        LEAD · SUB (subscriptions) · COMP (compliance) · TRAIN
             MSG · BILL · SYS · VEND

Twenty objs  camelCase singular: serviceSubscription, consentRecord
Twenty flds  camelCase: renewalDate, phishPronePercent
Credentials  vendor-purpose-env: brevo-transactional-prod
```

> **TAG EVERY WORKFLOW WITH ITS GOHIGHLEVEL ORIGIN**
>
> During migration, put the source GoHighLevel workflow ID in the n8n workflow description and in `automationRun.ghlWorkflowId`. When someone asks in month four whether the old renewal reminder was ever rebuilt, that field answers it in one query instead of a day of archaeology.

Part II · Core CRM

This is the part of the migration that works cleanly. Every capability in Part II is either native to Twenty or a short n8n workflow. If the project stopped here it would already have a functioning CRM — it would simply have no way to contact anyone.

7. Contacts, companies and the data model

7.1 Contacts → Twenty People TWENTY

What GoHighLevel does. Stores contacts with standard fields, unlimited custom fields, tags, DND flags, assigned user, source attribution, and full interaction history.

How to build it.

1. Map GoHighLevel standard fields to Twenty `Person`: name → `name` (full name), email → `emails`, phone → `phones`, company → relation to `Company`.
2. Create custom fields via the Metadata API, scripted and committed — not clicked through the UI. The workspace must rebuild in one command, because the local instance will be reset more than once.
3. Add `ghlContactId` (TEXT) to every migrated object. This is the idempotency key for the entire migration and for any later reconciliation.
4. Add `sourceSystem` and `lastSyncedAt` if any parallel-run period is planned.

```
POST /rest/metadata/fields
Authorization: Bearer <TWENTY_API_KEY>

{ "objectMetadataId": "<person-object-id>",
  "name": "ghlContactId", "label": "GHL Contact ID",
  "type": "TEXT", "icon": "IconHash" }
```

Gotchas.

- **Custom field fill rate should be measured before migrating.** A field populated on 2% of contacts is a field to drop, not port. Measuring it requires reading contact records, which the audit's no-PII rule forbids — so use the structural proxy instead: a field referenced by no workflow, no form and no template is a drop candidate on those grounds alone.
- **DND flags do not migrate as flags.** They become `consentRecord` rows, because a boolean on a contact cannot express per-channel consent with a source and a timestamp. See §21.
- Twenty's `emails` and `phones` fields hold a primary plus additional values. Do not create separate custom fields for secondary contacts.

7.2 Companies / Businesses → Twenty Companies TWENTY

Direct mapping. Twenty `Company` covers name, domain, employees, address and arbitrary custom fields. Add `brand` as a SELECT (Aspire TSS / Aspire eLearning) — both brands share one workspace separated by this field, not two deployments. A client buying SOC services is a natural security-awareness-training prospect, and separate instances would hide exactly that.

7.3 Custom fields → Twenty fields TWENTY

GoHighLevel field type	Twenty type	Notes
Single line text	`TEXT`	—
Multi line text	`TEXT`	Twenty has long-text rendering; no separate API type needed for most uses
Numerical	`NUMBER`	—
Monetary	`CURRENCY`	Twenty stores amount + currency code
Date	`DATE` / `DATE_TIME`	Choose deliberately — reminders need time, renewals do not
Dropdown (single)	`SELECT`	Options need `label`, `value`, `position`, `color`
Multiple options / checkbox group	`MULTI_SELECT`	—
Checkbox (single)	`BOOLEAN`	—
Radio	`SELECT`	—
File upload	Attachment	Twenty attachments; the file itself may stay with the form vendor
Phone / Email	`PHONES` / `EMAILS`	Use the structured types, not TEXT
Signature	Attachment + `LINKS`	Real signatures belong with the e-sign vendor

7.4 Custom values → `mergeVariable` object TWENTY+N8N

What GoHighLevel does. Account-level variables (business name, booking link, support email) available as merge tags across templates and workflows.

Gap. Twenty has no account-level variable store.

How to build it. Create a `mergeVariable` object with `key`, `value` and `scope` (Global / Aspire TSS / Aspire eLearning). n8n reads it once per execution and caches for the run. Keep it in Twenty rather than n8n environment variables so a non-engineer can change the support email without a deployment.

7.5 Tags → `crmTag` object TWENTY

A GoHighLevel tag list is rarely just a vocabulary. In practice it accumulates three distinct things, and migrating them identically is a mistake:

What the tag really is	Example	Migrate as
A descriptive attribute	`cmmc-interested`	Keep as a tag record or a MULTI_SELECT field
Workflow state	`nurture-step-3`, `waiting-for-reply`	**A field.** State encoded as tags is invisible, unqueryable and drifts. Convert it.
Dead weight	`webinar-nov-2023`	Drop

The `crmTag.migrationAction` field exists to force this decision per tag: Keep as Tag, Convert to Field, or Drop. Do not migrate the tag list wholesale.

7.6 Custom objects → Twenty custom objects TWENTY

A clear upgrade. Twenty's custom objects are first-class: full REST/GraphQL endpoints, relations, views, permissions and workflow triggers, identical to standard objects. Aspire's model adds four business objects that GoHighLevel could only approximate with custom fields on deals:

Object	Why it exists	Key fields
`serviceSubscription`	SOCaaS/SECaaS are subscriptions, not one-off deals. Without this, renewal exposure and MRR are invisible.	serviceLine, status, startDate, renewalDate, mrr, termMonths, autoRenew
`complianceEngagement`	CMMC has defined levels and FCI/CUI scope. A text field loses the ability to filter, report or trigger renewal work.	framework, cmmcLevel, dataScope, status, auditDate, nextReviewDate
`trainingAccount`	A-SAT is priced per seat; consumption is the upsell signal and cannot live on a Person.	seatsPurchased, seatsConsumed, deploymentType, renewalDate
`phishingBaseline`	Phish-prone score becomes an account-health signal driving renewal conversations.	testDate, phishPronePercent, deltaVsPrevious, usersTested

8. Opportunities and pipelines

8.1 Pipelines and stages TWENTY

Direct mapping to Twenty `Opportunity` with a stage field. Build one pipeline at a time and confirm stage semantics with whoever runs the pipeline — GoHighLevel pipelines accumulate stages that mean nothing to anyone still employed.

Migration note. GoHighLevel supports multiple pipelines with independent stage sets. Twenty models stage as a SELECT on Opportunity, so multiple pipelines with genuinely different stages need either a `pipeline` SELECT plus a superset of stages, or separate objects. Prefer the former unless the pipelines share no vocabulary at all.

8.2 Opportunity value and forecasting TWENTY+N8N

Twenty holds amount, close date, stage and probability if added as a field. Its forecasting is weaker than GoHighLevel's and considerably weaker than Salesforce's.

> **CONFIRM BEFORE CALLING THIS A GAP**
>
> Weighted-pipeline forecasting is frequently listed as a missing feature and rarely used in practice. Check whether anyone actually opens the GoHighLevel forecast view before building a replacement. If the real requirement is "MRR by service line next quarter", that is a Twenty view over `serviceSubscription`, not a forecasting engine.

8.3 Pipeline automation TWENTY

GoHighLevel's Pipeline Stage Changed and Opportunity Status Changed triggers map directly to Twenty's **Record is updated** trigger scoped to the stage field. Field-scoping matters: an unscoped update trigger fires on every edit to every opportunity, including edits made by the workflow itself.

8.4 Stale opportunities TWENTY+N8N

GoHighLevel has a dedicated Stale Opportunities trigger. Twenty has no equivalent, because "nothing happened for 14 days" is an absence, and absences do not raise events.

Build: n8n Schedule Trigger (daily) → HTTP Request to Twenty searching opportunities where `stage` is open and `updatedAt` < now − 14 days → Loop Over Items → notify owner → write an `automationRun` record. This pattern — **scheduled query for an absence** — replaces every GoHighLevel trigger that is really a time-based condition, and appears repeatedly in §12.

9. Segmentation, bulk actions, import and export

GoHighLevel	Replacement	How
Smart lists / saved filters	TWENTY	Twenty saved views with filters and sorts, shareable across the workspace. Direct equivalent.
Bulk edit	TWENTY	Native multi-select and edit in table view.
Bulk add to workflow	TWENTY+N8N	Twenty's **Manual trigger in bulk mode** runs a workflow over selected records — the closest equivalent, and better than GoHighLevel's because it is explicit rather than a silent enrolment.
Bulk SMS / email from a list	VENDOR	Segment exported from Twenty by n8n, handed to the ESP or CPaaS. Consent check is mandatory (§21).
CSV import	TWENTY	Native import with field mapping.
CSV export	TWENTY	Native export from any view.
Duplicate detection / merge	TWENTY+N8N	Twenty has basic duplicate detection. Robust dedup belongs in n8n **at ingest** — search by email before create, upsert rather than blind create. Cleaning duplicates afterwards is far more expensive than not creating them.
Manual actions queue	TWENTY+N8N	Twenty Tasks assigned to a user reproduce the queue. The click-to-dial that made it useful comes from the CPaaS, not the CRM.
Engagement score	TWENTY+N8N	A NUMBER field on Person, recalculated by a scheduled n8n workflow from `messageLog`, `appointment` and `formSubmission` counts. More transparent than GoHighLevel's opaque score.

10. Users, roles and permissions

GoHighLevel	Replacement	How
User accounts	TWENTY	Workspace members. Count active seats during the audit — dormant GoHighLevel users are common.
Roles and permissions	TWENTY	Object-level roles. Field-level restriction is thinner than GoHighLevel's; verify against any genuine need to hide specific fields.
Teams	TWENTY	Confirm whether teams drive routing logic — if so, that logic moves to n8n and teams become a field.
SSO	VENDOR	Twenty SSO is an Organization-tier feature. Confirm whether Aspire requires it before treating this as blocking.
API keys	TWENTY	Settings → API & Webhooks. Scope per role. Rotate on a schedule.

PERMISSIONS DESERVE A DESIGN PASS, NOT A COPY

Aspire holds client security-posture data and sells compliance services. The CRM should meet the standard Aspire advises clients to meet. Migrate roles as a deliberate design — least privilege, field-level restriction on anything sensitive, admin actions logged — rather than replicating whatever accumulated in GoHighLevel.

Part III · Automation

This part carries the migration. Everything else is data structure or a vendor contract; this is where the business logic lives, and where the risk of silent loss is highest.

AUTOMATION LOGIC DOES NOT MIGRATE AUTOMATICALLY

No tool migrates workflows between CRM platforms. Every GoHighLevel workflow must be read, understood and rebuilt individually. Worse, **GoHighLevel's public API exposes workflow metadata only** — id, name, status, dates. Step and action internals are not available on any documented endpoint, a limitation confirmed by a feature request that has sat unresolved since 2022. The inventory must therefore be built by reading each workflow, and anything missed dies silently at cutover and is noticed weeks later.

11. Automation architecture: Twenty workflows vs n8n

Both engines are available. Using the wrong one produces either an unmaintainable Twenty workflow or a trivial n8n workflow that adds a network hop to a field update.

11.1 The decision rule

Use Twenty workflows when	Use n8n when
Everything happens inside Twenty	Anything external is touched — a vendor, an API, a file
One or two steps	More than two conditional branches
Simple field update on record change	Loops or batching
Creating a related record	Retry, error handling or a dead-letter path matters
A manual action a user launches from a record	The logic will change often
A simple scheduled tidy-up	Human approval is needed mid-flow
	An LLM is involved
	Data must be transformed, not just copied

DEFAULT TO N8N WHEN GENUINELY UNSURE

n8n workflows are versioned in Git, testable in isolation, observable per execution and recoverable on failure. Twenty workflows are none of those things. The cost of an unnecessary n8n workflow is one network call; the cost of business logic trapped in an unversioned, unobservable Twenty workflow is discovered during the next migration.

11.2 The pattern that covers most cases

```
Twenty record changes
    │
    ▼  webhook (HMAC-signed)
n8n Webhook node
    │
    ▼  verify signature — reject if invalid
Code node: HMAC SHA256 over "${timestamp}:${payload}"
    │
    ▼  filter early and exit cheaply
IF node: is this the event we care about?  ── no ──▶  NoOp (end)
    │ yes
    ▼  loop guard
IF node: was this change made by us?       ── yes ─▶  NoOp (end)
    │ no
    ▼
[ business logic ]
    │
    ▼  write results back
HTTP Request → Twenty Core API
    │
    ▼  observability
HTTP Request → create automationRun record
```

Twenty does not filter webhook events at source — every event type reaches a subscribed endpoint. The early filter is not an optimisation; without it every workflow wakes for every change in the workspace and the 100-requests-per-minute API ceiling arrives quickly.

12. Complete trigger mapping

Every GoHighLevel workflow trigger, with its replacement. Sourced from HighLevel's published trigger list plus the 2026 additions.

12.1 Contact triggers

GoHighLevel trigger	Replacement	Build
Contact Created	TWENTY	Twenty **Record is created** on Person. For externally-sourced contacts, n8n creates the record so the trigger fires downstream of ingest.
Contact Changed	TWENTY	Twenty **Record is updated**. **Scope it to specific fields** or it fires on every edit including the workflow's own writes.
Contact Tag (added/ removed)	TWENTY	If tags became a MULTI_SELECT field: Record is updated scoped to that field. If tags are `crmTag` relation records: Record is created/deleted on the join.
Birthday Reminder	N8N	Schedule Trigger daily → query Twenty for people whose birthday month/day matches today → loop. A date-match query, not an event.
Custom Date Reminder	N8N	Same pattern against any DATE field — renewal, audit date, next review. This single workflow, parameterised by field name, replaces every GoHighLevel date reminder.
Note Added / Note Changed	TWENTY	Record is created / updated on Note.
Task Added	TWENTY	Record is created on Task.

Task Reminder	N8N	Scheduled query for tasks due within the window. Absence-and-time conditions are always scheduled queries.

12.2 Contact action triggers

GoHighLevel trigger	Replacement	Build
Form Submitted	VENDOR → N8N	Form vendor posts to an n8n Webhook node. n8n validates, dedupes, upserts the Person, writes `formSubmission`, and creates a `consentRecord` if consent was captured.
Survey Submitted	VENDOR → N8N	Identical to Form Submitted.
Order Form Submission	VENDOR → N8N	Commerce vendor webhook → create `payment` and `invoice` records.
Customer Replied	VENDOR → N8N	ESP or CPaaS inbound webhook → n8n writes a `messageLog` with direction Inbound and status Replied. **An SMS reply of STOP must also write an opt-out `consentRecord`** — this is a legal obligation, not a nicety.
Trigger Link Clicked	N8N	n8n Webhook node serves the short URL, records the click on `trackedLink`, then responds with a 302 via Respond to Webhook. Fully replaceable with no vendor.
Twilio Validation Error	VENDOR → N8N	CPaaS error webhook → error workflow → dead-letter record.

12.3 Appointment, opportunity and event triggers

GoHighLevel trigger	Replacement	Build
Customer Booked Appointment	VENDOR → N8N	Scheduling vendor webhook → n8n creates an `appointment` record → Twenty Record-created trigger fires downstream logic.
Appointment Status (confirmed, cancelled, showed)	TWENTY	Once the `appointment` record exists, status changes are a Twenty Record-is-updated trigger scoped to `status`.
Appointment No-Show (2026)	N8N	Scheduled query: appointments where `scheduledAt` is past and status is still Booked. A no-show is an absence — no vendor emits it reliably.
Opportunity Status Changed	TWENTY	Record is updated on Opportunity, scoped to status.
Pipeline Stage Changed	TWENTY	Record is updated, scoped to stage.
Stale Opportunities	N8N	Scheduled query on `updatedAt` older than threshold (§8.4).
Call Status	VENDOR → N8N	CPaaS webhook → `callLog` record with outcome and duration.
Email Events (open, click, bounce, complaint)	VENDOR → N8N	ESP webhook → update `messageLog` status. **Bounce and complaint must also write a `consentRecord`**, or hard-bounced addresses are retried forever and reputation degrades.

12.4 Commerce, membership and web triggers

GoHighLevel trigger	Replacement	Build
Invoice (sent / paid / overdue)	VENDOR → N8N	Finance system webhook → update the Twenty `invoice` record. Overdue is an absence: scheduled query on `dueDate` past with status not Paid.
Subscription (created / trial→active / cancelled, 2026)	TWENTY+N8N	Billing webhook → update `serviceSubscription.status` → Twenty Record-updated trigger drives downstream logic. This is the highest-value trigger for Aspire's recurring-revenue model.
Membership New Signup	VENDOR → N8N	LMS webhook → `membershipEnrollment` record. Likely aspireelearning.com.
Offer / Product Access Granted or Removed	VENDOR → N8N	LMS webhook → update enrolment status.
Product Completed / Category Completed	VENDOR → N8N	LMS completion webhook → update `progressPercent`, issue certificate.
User Login	VENDOR	LMS event. Verify anyone automates on this before rebuilding it.
Facebook Lead Form Submitted	N8N	n8n has a Facebook Lead Ads trigger; otherwise webhook via the Meta API.
Website Visit (page, time on site, scroll depth — 2026)	VENDOR → N8N	Requires site-side tracking. Analytics vendor or a script posting to an n8n webhook. **Check actual usage first** — this is frequently enabled and rarely automated on.
Shopify: Abandoned Checkout / Order Placed / Order Fulfilled	N8N	n8n has a Shopify node with these triggers. Direct replacement if Aspire uses Shopify at all.

> **THE PATTERN BEHIND THE WHOLE TABLE**
>
> Three rules cover every trigger without exception. **(1)** If the event happens inside Twenty, use a Twenty record trigger, always field-scoped. **(2)** If it happens in a vendor, the vendor's webhook hits n8n, n8n writes a Twenty record, and a Twenty trigger fires downstream. **(3)** If the "event" is really an absence or an elapsed time — stale, overdue, no-show, birthday, reminder — it is a **scheduled query**, because nothing emits an event for something that did not happen.

13. Complete action mapping

All fourteen GoHighLevel workflow action categories, every action, with its replacement. This is the checklist that proves nothing was skipped.

13.1 Contact Actions

GoHighLevel action	Replacement	Build
Create Contact	TWENTY	Twenty **Create Record**, or n8n `POST /rest/people`.
Find Contact	TWENTY	Twenty **Search Records** (200-result cap), or n8n GET with a filter.

Update Contact Field	TWENTY	Twenty **Update Record**.
Add / Remove Contact Tag	TWENTY	Update the MULTI_SELECT field, or create/delete the `crmTag` relation.
Assign to User / Remove Assigned User	TWENTY	Update the owner relation. Round-robin assignment logic lives in n8n (§15.1).
Add Note	TWENTY	Create a Note record linked to the target.
Add Task	TWENTY	Create a Task with assignee and due date.
Copy Contact	N8N	GET then POST with modified fields. Rare — confirm it is used.
Delete Contact	TWENTY	Twenty **Delete Record**. Prefer a soft-delete field for anything auditable.
Disable / Enable DND	TWENTY+N8N	Create a `consentRecord` with the appropriate status. **Not a boolean flag** — per-channel state with a source and timestamp is what makes it defensible (§21).
Edit Conversation	DROP	Tied to the GoHighLevel inbox. No equivalent and no replacement needed once the inbox architecture is settled (§17).
Modify Contact Engagement Score	TWENTY+N8N	Update a NUMBER field. Prefer recomputing on a schedule over incrementing in-flight — incremental scores drift and cannot be reconstructed.
Add / Remove Contact Followers	TWENTY	A relation to workspace members, or a MULTI_SELECT.

13.2 Communication Actions

This is the category that generates most of the vendor requirement. Every action here needs a channel Twenty does not have.

GoHighLevel action	Replacement	Build
Send Email (1:1, transactional)	TWENTY+N8N	Twenty **Send Email** works for genuine one-to-one from a synced mailbox. Prefer routing through the ESP sub-workflow anyway, so all sending is consent-checked and logged in one place.
Send Email (templated / at volume)	VENDOR	ESP via n8n. Consent check → merge fields → send → log `messageLog`. See §18.
Send SMS	VENDOR	CPaaS via n8n (Twilio node or HTTP Request). §19.
WhatsApp	VENDOR	CPaaS WhatsApp Business API. The n8n Twilio node has a WhatsApp toggle. Template pre-approval is required by Meta — a lead time, not a build task.
Call (auto-dial / connect)	VENDOR	CPaaS voice API. §20.
Messenger / Instagram DM	VENDOR	Meta APIs via n8n HTTP Request. Confirm real usage before building — B2B security firms rarely sell through Instagram DM.
Facebook / Instagram Interactive Messenger	VENDOR	As above, with quick-reply payloads.
Reply in Comments	VENDOR	Meta Graph API. Usually droppable.

GMB Messaging	VENDOR	Google Business Profile API.
Send Live Chat Message	VENDOR	Requires a chat widget vendor.
Send Slack Message	N8N	n8n Slack node. **Direct replacement, no vendor gap** — Slack is already in use.
Send Internal Notification	N8N	Slack node or SMTP. Internal notification never needs the marketing ESP.
Manual Action (queue a call/ SMS task)	TWENTY	Create a Twenty Task assigned to the rep.
Send Review Request	VENDOR	Templated email or SMS with a review link, via the ESP/CPaaS sub-workflow. §27.
Conversation AI	N8N	n8n AI Agent node with tools, routed through the LLM gateway. Needs a channel to speak on. §26.

13.3 Internal Tools Actions

This category is where n8n is not merely equivalent but materially better.

GoHighLevel action	Replacement	Build
If / Else	N8N	n8n **IF** (two branches) or **Switch** (many). Twenty's Filter handles only simple cases. n8n supports arbitrary expressions.
Wait Step	TWENTY+N8N	Twenty **Delay** for simple waits; n8n **Wait** for duration, specific time, or **until a webhook is called** — the last has no GoHighLevel equivalent and enables true approval gates.
Goal Event	N8N	No native equivalent in either tool. Rebuild as an explicit check before each step: query current state, exit if the goal is met. More verbose than GoHighLevel, and more legible — the exit condition is visible rather than implied.
Split (A/B percentage)	N8N	Code node emitting a random bucket → Switch. Deterministic hashing on contact ID is better than random if the same contact must stay in the same bucket across runs.
Go To	N8N	n8n has no arbitrary jump. Restructure into sub-workflows called by name — which is what a Go To was standing in for.
Remove from Workflow	N8N	NoOp on a branch, or **Stop and Error** for an explicit abort.
Arrays	N8N	Native. **Item Lists**, **Split Out**, **Aggregate**, **Loop Over Items**, or a Code node.
Drip Mode (throttled release)	N8N	**Loop Over Items** with a batch size plus a Wait between batches. Also the correct tool for staying inside Twenty's 100-requests-per-minute ceiling.
Text Formatter	N8N	Expressions or a Code node. Full JavaScript string handling rather than a fixed formatter list.
Custom Code	TWENTY+N8N	n8n **Code** node (JavaScript or Python); Twenty's **Code** action for in-CRM logic. Both exceed GoHighLevel's sandbox.
Update Custom Value	TWENTY+N8N	Update the `mergeVariable` record via the Core API.

Math Operation	N8N	Expression or Code node.

13.4 Send Data, Appointments, Opportunities

GoHighLevel action	Replacement	Build
Webhook / Custom Webhook	TWENTY+N8N	Twenty **HTTP Request** action, or n8n HTTP Request with retry and error routing. Prefer n8n where the target is unreliable.
Google Sheets	N8N	n8n Google Sheets node. Direct replacement. **Worth auditing what those sheets do** — a Sheets step often marks a process that should have been a CRM object.
Update Appointment Status	TWENTY	Update Record on `appointment`. Push back to the scheduling vendor if it is the source of truth for the slot.
Generate One Time Booking Link	VENDOR	Scheduling vendor API returns a single-use link; n8n injects it into the message.
Create / Update Opportunity	TWENTY	Create or Update Record. Use **Upsert** where a duplicate is possible.
Remove Opportunity	TWENTY	Delete Record, or set a closed status — usually preferable for reporting continuity.

13.5 Payments, Marketing, Affiliate, Courses, Communities

GoHighLevel action	Replacement	Build
Stripe One-Time Charge	VENDOR	Payment processor API via n8n → write a `payment` record. **Charging a card from a workflow deserves an approval gate**, not a silent automation.
Send Invoice	VENDOR	Finance system API → `invoice` record in Twenty.
Send Documents and Contracts	VENDOR	E-sign vendor API → `quote` record with `documentUrl`; completion webhook updates status.
Add to Google Analytics	N8N	GA4 Measurement Protocol via HTTP Request.
Add to Google AdWords	N8N	Google Ads API — offline conversion import.
Add / Remove Custom Audience (Facebook)	N8N	Meta Marketing API. **Hash email addresses before upload** as the API requires.
Facebook Conversion API	N8N	Meta CAPI via HTTP Request. Server-side conversion tracking survives the migration unchanged.
Add to Affiliate Manager · Update Affiliate · Add/ Remove from Affiliate Campaign	VENDOR / DROP	No Twenty equivalent. **Check whether Aspire runs an affiliate programme at all** — for a B2B compliance firm this is usually an unused GoHighLevel module, and the honest disposition is DROP.
Course Grant Offer / Revoke Offer	VENDOR	LMS API via n8n → update `membershipEnrollment`. Likely aspireelearning.com.
Grant / Revoke Group Access	VENDOR / DROP	Communities module. Verify usage — frequently enabled, rarely used.

13.6 IVR, AI and Eliza actions

GoHighLevel action	Replacement	Build
Gather Input on Call · Play Message · Connect to Call · End Call · Record Voicemail	VENDOR	The whole IVR category is CPaaS call-control. Rebuilt as a call-flow definition in the CPaaS (TwiML or equivalent) with n8n providing dynamic decisions over HTTP. §20.
AI Prompt (Workflow AI)	N8N	n8n **Basic LLM Chain** for deterministic single-prompt tasks; **AI Agent** where runtime tool use is genuinely needed. Route through the LLM gateway for cost caps and audit. §26.
Eliza AI Appointment Booking	N8N + VENDOR	AI agent with scheduling-vendor tools. Substantial build — size it honestly rather than assuming parity.
Send to Eliza Agent Platform	N8N	Hand off to an n8n AI agent workflow.

> **EXPORT EVERY AI PROMPT BEFORE THE ACCOUNT CLOSES**
>
> Voice AI and Conversation AI prompts are written assets that took real iteration to get right. They are not recoverable after termination and no export exists. **Copy them verbatim during the manual sweep**, regardless of whether those features are being rebuilt — the prompt is worth keeping even if the platform is not.

14. The n8n foundation

Five things must exist before the first business workflow is built. Retrofitting any of them across fifty workflows is painful enough that teams skip it and live with the consequences.

14.1 Webhook signature verification

Twenty signs every webhook with HMAC SHA256 over `${timestamp}:${payload}`, delivered in the `x-twenty-webhook-timestamp` and `x-twenty-webhook-signature` headers. An n8n webhook URL is a public endpoint; without verification anyone who learns the URL can inject records.

```
// Code node, immediately after the Webhook node
const crypto = require('crypto');

const secret    = $env.TWENTY_WEBHOOK_SECRET;
const timestamp = $input.first().json.headers['x-twenty-webhook-timestamp'];
const signature = $input.first().json.headers['x-twenty-webhook-signature'];
const payload   = JSON.stringify($input.first().json.body);

const expected = crypto
  .createHmac('sha256', secret)
  .update(`${timestamp}:${payload}`)
  .digest('hex');

// Constant-time compare — a plain === leaks timing information
const ok = crypto.timingSafeEqual(
  Buffer.from(expected), Buffer.from(signature));

if (!ok) throw new Error('Invalid webhook signature');

// Reject replays: refuse anything older than five minutes
if (Math.abs(Date.now() - Number(timestamp)) > 300000) {
  throw new Error('Webhook timestamp outside tolerance');
}

return $input.all();
```

14.2 The error workflow

n8n provides three layered mechanisms. Use all three; they solve different failures.

Mechanism	Setting	Use for
Retry on Fail	Node → Settings → Retry On Fail, with attempts and wait	Transient failures: timeouts, 429s, brief vendor outages. Two to three retries is standard.
On Error → Continue	Node → Settings → On Error	Non-critical steps that must not block the run. Options are Stop Workflow (default), Continue, and Continue using error output.
Error Trigger workflow	A separate workflow starting with the Error Trigger node, set as the error workflow on every production workflow	Workflow-level catch-all. Fires after the whole workflow fails.

Build the error handler once:

```
[SYS] Error Handler — Error Trigger

Error Trigger
   ↓
Code: extract workflow name, execution id, node, message, contact id
   ↓
HTTP Request → Twenty: create automationRun
   { workflowName, status: "FAILED", errorMessage,
     n8nExecutionId, startedAt }
   ↓
IF: is this a repeat failure of the same workflow within 1 hour?
   ├─ yes → NoOp        (suppress alert storms)
   └─ no  → Slack: post to #crm-alerts with a link to the execution
```

> **A SILENT FAILURE IS WORSE THAN AN OUTAGE**
>
> An outage is noticed. A workflow that quietly stopped enrolling leads is discovered when someone asks why the pipeline is thin, typically three weeks later. GoHighLevel gave no visibility into this at all. n8n does — but only if the error workflow is wired to every production workflow from the beginning.

14.3 Idempotency and loop prevention

Two failure modes will corrupt data if not designed for up front.

Idempotency. Webhooks retry. A vendor that does not get a 200 quickly will send the same event again, and a workflow that blindly creates records will produce duplicates. Always search-then-upsert on a deterministic external key:

```
1. HTTP Request → Twenty: GET /rest/people?filter=emails.primaryEmail[eq]:<email>
2. IF found:  PATCH /rest/people/<id>
   ELSE:      POST  /rest/people
```

Where the object carries an external id — `ghlContactId`, `vendorEventId`, `n8nExecutionId` — filter on that instead of email. It is stable; email is not.

Loop prevention. A Twenty record-updated webhook triggers n8n, n8n writes back to Twenty, the write fires the webhook again. Left unguarded this exhausts the 100-request-per-minute ceiling within minutes and can corrupt records on both sides.

Guard	Implementation
Field-scoped triggers	**Strongest guard.** Scope the Twenty trigger to fields the workflow never writes. If the workflow writes `score`, do not trigger on `score`.
`lastSyncedAt` check	n8n sets it on write; the workflow exits early if the record changed within a few seconds of that timestamp.
`sourceSystem` field	n8n stamps its own writes and skips events carrying its own stamp.
Execution ceiling	An alert when one workflow exceeds an expected executions-per-hour threshold. The last line of defence, and the one that catches the case nobody predicted.

14.4 Sub-workflows — the structural decision that pays for itself

Every vendor call goes in a sub-workflow with a stable input contract. Business workflows call the sub-workflow; they never call the vendor directly.

```
[VEND] Send Email — Sub-workflow
  Input contract:
    { personId, templateKey, mergeData, channel: "email" }

  1. HTTP Request → Twenty: fetch person + consentRecords
  2. IF consent for channel is not "Opted In"  → return { sent:false,
                                                   reason:"no_consent" }
  3. HTTP Request → Twenty: fetch mergeVariable values
  4. HTTP Request → ESP: send with merged template
  5. HTTP Request → Twenty: create messageLog
  6. return { sent:true, vendorMessageId }
```

Three things follow from this, and all three matter:

- **Consent is enforced in exactly one place.** A workflow author cannot forget the check, because they never touch the sending API.
- **Swapping the ESP touches one workflow**, not fifty. Given that vendor selection is happening in parallel with the build, this is not hypothetical.
- **Every send is logged identically**, so `messageLog` is a complete record rather than a partial one.

14.5 Rate limiting against Twenty

Twenty allows **100 requests per minute** and batches of **60 records**. A migration or nightly job that loops one record at a time will hit the ceiling immediately.

Pattern	Use
Loop Over Items, batch size 50, Wait 1 s between batches	Any bulk operation. Keeps well inside the ceiling with headroom.
Batch create/update, 60 records per call	Migration loads and nightly syncs.
GraphQL batch upsert	Create-or-update in one call — the most efficient migration path.
Retry On Fail with backoff on 429	Every node touching Twenty.

15. Worked recipes

Six automations built end to end. Between them they exercise every pattern in §12–13; most GoHighLevel workflows are a variation on one of these.

15.1 Inbound lead → qualified, assigned, acknowledged

GoHighLevel equivalent: Form Submitted → Create Contact → Add Tag → Assign to User → Send Email → Send Internal Notification.

```
[LEAD] Create Person — Web Form

 1  Webhook (POST from form vendor)
 2  Code: verify vendor signature
 3  Set: normalise — lowercase email, E.164 phone, trim
 4  HTTP Request → Twenty: GET /rest/people?filter=emails.primaryEmail[eq]:{{email}}
 5  IF exists
      ├ yes → PATCH person (fill blanks only, never overwrite)
      └ no  → POST person
 6  IF company domain present
      → search Company by domain; upsert; link person
 7  POST formSubmission  { payload, sourceUrl, consentGiven }
 8  IF consentGiven
      → POST consentRecord { channel:"Email", status:"Opted In",
                             source:"Form Submission", effectiveAt:now }
 9  Code: score the lead
      +30 corporate domain · +20 CMMC/SOC in message
      +25 employees > 50   · −40 free email domain
10  PATCH person { leadScore }
11  Switch on score
      ├ ≥60 → assign to senior rep   (round-robin, §below)
      ├ 30–59 → assign to SDR queue
      └ <30 → tag nurture, no assignment
12  POST Task { assignee, "Call new lead", due +1 business day }
13  Execute Sub-workflow [VEND] Send Email
      { templateKey:"lead_ack", personId, mergeData }
14  Slack → #sales: name, company, score, link to Twenty record
15  POST automationRun { status:"Success" }
```

Round-robin assignment has no native equivalent in either tool. Deterministic and stateless:

```
// Code node — no counter to store, no drift
const reps = ['rep-uuid-1','rep-uuid-2','rep-uuid-3'];
const key  = $json.personId;                    // stable per person
const hash = [...key].reduce((a,c) => a + c.charCodeAt(0), 0);
return [{ json: { assigneeId: reps[hash % reps.length] } }];
```

Hashing the person id rather than incrementing a counter means the same person always routes to the same rep on re-runs, and no state has to live in n8n.

15.2 Multi-step nurture sequence with exit conditions

GoHighLevel equivalent: Wait → Send Email → If/Else → Goal Event → Drip Mode.

GoHighLevel's Goal Event silently removes a contact when a condition is met. n8n has no equivalent, so the exit check is written explicitly before each send. More verbose, and considerably easier to reason about — the exit condition is visible rather than implied by a setting on another node.

```
[LEAD] Nurture Sequence — 5 touches over 21 days

Trigger: Twenty webhook — person.updated where tag = "nurture"

 1  Verify signature · filter event · loop guard
 2  Sub-workflow [VEND] Send Email  { templateKey:"nurture_1" }
 3  Wait 3 days
 4  ── EXIT CHECK (repeat before every subsequent send) ──
    GET person + opportunities + appointments
    IF   opportunity created
      OR appointment booked
      OR consentRecord status = "Opted Out"
      OR tag "nurture" removed
      → PATCH person { nurtureStatus: "Exited" }
        POST automationRun { status:"Success", note:"goal met" }
        Stop and Error (explicit end)
 5  Sub-workflow [VEND] Send Email  { templateKey:"nurture_2" }
 6  Wait 5 days → EXIT CHECK → touch 3
 7  Wait 7 days → EXIT CHECK → touch 4
 8  Wait 6 days → EXIT CHECK → touch 5
 9  PATCH person { nurtureStatus:"Completed" }
```

LONG WAITS AND WORKFLOW EDITS

A 21-day sequence means executions sit waiting for three weeks. Editing the workflow while executions are in flight produces undefined behaviour on resume. For sequences longer than a few days, prefer a **scheduled state-machine**: store `nurtureStep` and `nextTouchAt` on the person, and run a daily workflow that processes whoever is due. It survives edits, restarts and redeploys — none of which a long Wait does.

15.3 Appointment reminder with reschedule

GoHighLevel equivalent: Customer Booked Appointment → Wait → Send SMS → Send Email.

```
[MSG] Appointment Reminders — Schedule

Trigger: Schedule, hourly

1  GET appointments where scheduledAt in next 24–25 h
                     AND status = "Booked"
                     AND reminder24hSent = false
2  Loop Over Items (batch 50, wait 1 s)
3    Sub-workflow [VEND] Send Email { templateKey:"appt_reminder_24h" }
4    IF person has mobile AND SMS consent = Opted In
       → Sub-workflow [VEND] Send SMS { templateKey:"appt_sms_24h" }
5    PATCH appointment { reminder24hSent: true }

Second schedule at 2 h uses the same shape with reminder2hSent.
```

The `reminderSent` boolean is what makes the hourly schedule safe to re-run. Without it, a retry sends the reminder twice.

15.4 Renewal escalation — the highest-value automation for Aspire

No GoHighLevel equivalent, because GoHighLevel has no subscription object. This is the automation the new data model makes possible.

```
[SUB] Renewal Escalation — Schedule

Trigger: Schedule, daily 06:00 UTC

1  GET serviceSubscription where status = "Active"
       AND renewalDate within next 90 days
2  Switch on days-until-renewal
   ├ 90 → PATCH status "Renewal Due"
   │      POST Task to owner: "Begin renewal conversation"
   │      Slack → #renewals
   ├ 60 → Sub-workflow [VEND] Send Email { "renewal_60d" }
   │      POST Task: "Confirm renewal intent"
   ├ 30 → IF no opportunity linked to this subscription
   │        → POST Opportunity { stage:"Renewal", amount: mrr × term }
   │        → Slack → #renewals ESCALATION
   └  7 → IF still no opportunity
            → Slack → #sales-leadership URGENT
            → POST Task to manager
3  POST automationRun
```

This is worth building first. It is pure business value, touches no vendor, needs no channel beyond internal Slack, and can be built and proven **before any vendor decision is made** — which makes it the ideal first workflow to demonstrate the new stack.

15.5 Human approval gate

No GoHighLevel equivalent at all. n8n's Wait node exposes `$execution.resumeUrl`, generated at runtime, which turns any workflow into an approval flow.

```
[MSG] Campaign Approval

1  Schedule or manual trigger
2  Build the campaign segment from Twenty
3  Code: build approve / reject URLs from $execution.resumeUrl
4  Slack: post summary + recipient count + Approve / Reject buttons
5  Wait — On Webhook Call  (resumes when a button is clicked)
6  IF approved
     ├ yes → Loop batches → [VEND] Send Email → POST campaign record
     └ no  → Slack: "Campaign rejected by {{user}}" → end
```

Use this for anything with blast radius: bulk sends, card charges, bulk record updates, anything that touches more than a handful of contacts.

15.6 Inbound reply handling and consent

GoHighLevel equivalent: Customer Replied trigger.

```
[MSG] Inbound Reply Handler

1  Webhook (ESP or CPaaS inbound)
2  Verify signature
3  Match sender → Twenty person (email or E.164 phone)
4  POST messageLog { direction:"Inbound", status:"Replied", channel }
5  Switch on channel + content
   ├ SMS body matches /^(STOP|UNSUBSCRIBE|CANCEL|QUIT)$/i
   │   → POST consentRecord { channel:"SMS", status:"Opted Out",
   │                          source:"Reply STOP", effectiveAt:now }
   │   → Slack → #compliance
   ├ Email is an auto-reply / OOO  → NoOp
   └ genuine reply
       → PATCH person { lastRepliedAt: now }
       → POST Task to owner: "Reply received — respond"
       → Slack → owner's DM
```

THE STOP BRANCH IS A LEGAL REQUIREMENT

Carrier rules require honouring an SMS opt-out immediately. This branch is not an enhancement to add later — if SMS is in scope, it ships with the first SMS send or the SMS programme should not launch.

16. Workflow migration procedure

Because GoHighLevel's API does not expose workflow internals, the inventory is manual. Structure it so it is done once, correctly.

16.1 Per-workflow record

For every GoHighLevel workflow, capture:

Field	Notes
Name, id, status	From `GET /workflows/` — the one thing the API does provide
Trigger(s)	Map to §12
Step sequence	Every action in order, with its settings — wait durations, template ids, conditions
Enrolment volume	**Not available via API.** Read from the execution-log view in the UI
Last execution	Same source
Decision	Rebuild · Merge into another · Drop
Rebuilt as	n8n workflow name once built

16.2 The discard pass

Run this **before** capturing step detail. It typically removes a third to a half of the workflows, and each removed workflow is one nobody has to read, rebuild, test or explain.

- Draft and never published → drop
- Zero enrolments in 90 days → drop candidate
- Duplicates of another workflow → merge
- Depends only on a feature already classified DROP → drop

> **USE A 365-DAY WINDOW FOR ANYTHING ANNUAL**
>
> Aspire sells annual compliance engagements and yearly training renewals. A workflow named CMMC Annual Review or Renewal 90d Notice will show zero enrolments in a 90-day window and look dead. Re-check anything named like a renewal, review, annual or compliance cycle at 365 days before dropping it. This is the most likely way a genuinely important automation gets deleted.

16.3 Rebuild order

Order	Category	Why
1	Internal-only (Slack, tasks, field updates)	No vendor dependency. Build and prove the stack while vendor selection is still running. §15.4 is the model.
2	Twenty-internal record automations	Native Twenty workflows. Fast.
3	Inbound capture (forms, webhooks)	Needs the form vendor, but not the ESP.
4	Email-dependent	Blocked on ESP selection and domain warm-up.
5	SMS / voice-dependent	Blocked on CPaaS, A2P registration and number porting — the longest lead times.

16.4 Verification before cutover

For each rebuilt workflow, prove three things and record the evidence:

1. **It fires** — trigger the condition on a test record and confirm the execution.
2. **It does the same thing** — compare output against the GoHighLevel workflow's behaviour on the same input, field by field.
3. **It fails safely** — force an error and confirm the error workflow catches it, the alert arrives, and an `automationRun` record is written.

Then spot-check five rebuilt workflows by hand against the original GoHighLevel definitions. If all five match, trust the rest. If any is wrong, fix the method and re-verify the batch — a classification that is 90% right is indistinguishable from one that is 100% right until go-live day.

Part IV · Conversations and channels

17. The inbox problem and its architecture

What GoHighLevel does. One inbox showing every conversation with a contact across SMS, email, WhatsApp, Facebook, Instagram, Google Business Messages, live chat and calls — threaded, searchable, with the contact record beside it. A rep works the whole day from that screen.

The gap. Twenty has no inbox. Email threads sync to records via a connected mailbox; nothing else appears anywhere. This is the single most-felt daily loss in the migration, and it deserves an explicit decision rather than a hope that nobody notices.

17.1 Three options, honestly compared

Option	What it is	Cost	Best when
A. Record-only	All messages become `messageLog` records on the person. Full history, searchable, reportable. No live conversation view; replying means opening the vendor tool.	Lowest — included in the build. No extra vendor.	Conversations are low-volume, mostly email, and reps already live in Outlook or Gmail.
B. Shared inbox vendor	A dedicated shared-inbox or helpdesk product handles live conversation; n8n mirrors messages into Twenty as `messageLog`.	Additional vendor and per-seat cost.	Reps genuinely work conversationally across channels all day.
C. Channel-native	Email in the mail client, SMS in the CPaaS console, social in Meta Business Suite. Twenty holds the record.	Zero additional cost; highest context-switching burden.	Each channel has low volume and a single owner.

> **DECIDE THIS FROM EVIDENCE, NOT PREFERENCE**
>
> The audit's conversation histogram answers it directly: it shows how many conversations exist per channel over 90 days. If the mix is 95% email with a handful of SMS threads, **Option A is correct and costs nothing**. If there are thousands of live SMS conversations across several reps, Option A will be rejected by users in week two and Option B was always the answer.
>
> Do not choose before that number exists — and note that it is one of the few genuinely reversible decisions here, since `messageLog` is built either way.

17.2 What is built regardless

Under all three options, every message becomes a `messageLog` record: channel, direction, status, subject, vendor, vendor message id, timestamp, linked person. That preserves the thing that actually matters for a CRM — **the history is on the record, in one place, for reporting and for the next person who picks up the account.** What varies between options is only where a human types the reply.

18. Email

18.1 The two kinds of email, and why they must be separated

Kind	Examples	Replacement
Transactional / 1:1	Lead acknowledgement, appointment reminder, quote sent, password-style notices	`TWENTY+N8N` — Twenty's Send Email works from a synced mailbox; routing through the ESP sub-workflow is still preferable so consent and logging are uniform
Bulk / campaign	Newsletter, nurture sequences, announcements, event invitations	`VENDOR` — ESP required. No alternative exists

> **NEVER SEND BULK FROM A SYNCED MAILBOX**
>
> It is technically possible to loop Twenty's Send Email action over a list. Do not. A personal or shared mailbox has a low daily sending cap, no suppression list, no bounce handling and no reputation management. Sending campaign volume through it will get the mailbox rate-limited, then the sending domain flagged, and it puts **every** email Aspire sends — including genuine one-to-one sales conversations — at risk. The separation between transactional and bulk is not stylistic.

18.2 ESP requirements

Requirement	Why it matters here
Transactional + marketing on one platform	Two vendors means two suppression lists, and a contact who unsubscribes from one still receives the other
API with template merge	n8n passes `{ templateKey, mergeData }`; the ESP renders. Keeps HTML out of workflows
Suppression list with API access	Must import the GoHighLevel export and stay synchronised with `consentRecord`
Bounce / complaint / unsubscribe webhooks	Without these, `consentRecord` drifts out of date within weeks
Dedicated sending domain, DKIM/SPF/DMARC	Deliverability. Warm-up support matters as much as the feature itself
Open / click tracking with webhooks	Feeds engagement scoring and `campaign` statistics
Scheduled sending	Or n8n schedules and the ESP sends immediately — either is workable

18.3 Migration sequence for email

Order matters more here than anywhere else in the project.

Step	Action	Timing and risk
1	**Export the suppression list from GoHighLevel**	**Before anything else.** Verify the export actually works and the file is complete. This data is unrecoverable after termination
2	Select ESP	Critical path — everything below waits on it
3	Import suppression list into ESP **and** into `consentRecord`	Both. The ESP enforces at send; Twenty is the auditable record
4	Configure sending domain, DKIM, SPF, DMARC	DNS changes; coordinate with whoever owns DNS
5	**Begin domain warm-up**	**2–6 weeks.** Gradual volume ramp. Cannot be compressed
6	Rebuild templates	GoHighLevel's `editorData` cannot be read back via API — only exported HTML. Budget real rebuild time; this is routinely underestimated
7	Build the `[VEND] Send Email` sub-workflow	Can run in parallel with warm-up
8	Wire bounce / complaint / unsubscribe webhooks	Before first production send, not after
9	Send a low-volume real campaign	Verify deliverability with real recipients before committing
10	Cut over	Only once 1–9 are demonstrably complete

18.4 Trigger links and click tracking N8N

Fully replaceable with no vendor. n8n serves the redirect:

```
[MSG] Tracked Link Redirect

1  Webhook  GET /t/:slug        (Respond: Using Respond to Webhook node)
2  HTTP Request → Twenty: GET trackedLink where slug = :slug
3  IF not found → Respond 404
4  PATCH trackedLink { clickCount: +1 }
5  IF a person id is present in the query string
     → POST messageLog { status:"Clicked" }
     → PATCH person { lastEngagedAt: now }
6  Respond to Webhook: 302 → destinationUrl
```

Keep the redirect fast — do the Twenty writes after responding where the platform allows, or the click feels sluggish and some clients time out.

18.5 Email verification N8N

n8n plus a verification API, called at ingest in the lead workflow rather than in bulk later. Verifying before the first send protects sender reputation; verifying afterwards protects nothing.

19. SMS, WhatsApp and messaging channels

19.1 SMS VENDOR

What GoHighLevel does. SMS from a provisioned number, two-way threading in the inbox, templates, workflow sending, automatic STOP handling, A2P registration managed within the platform.

Replacement. A CPaaS provider driven by n8n. n8n has a Twilio node covering SMS, MMS and WhatsApp; any other provider works through HTTP Request.

```
[VEND] Send SMS — Sub-workflow
  Input: { personId, templateKey, mergeData }

1  GET person + consentRecords from Twenty
2  IF consent(channel="SMS") ≠ "Opted In"
     → return { sent:false, reason:"no_consent" }        ← hard stop
3  IF no mobile in E.164 format
     → return { sent:false, reason:"no_valid_number" }
4  Render template with mergeData
5  Twilio node (or HTTP Request) → send
6  POST messageLog { channel:"SMS", direction:"Outbound",
                     status:"Sent", vendorMessageId }
7  return { sent:true, vendorMessageId }
```

19.2 The three things that are not build tasks

Item	Lead time	Why it cannot be compressed
A2P 10DLC registration	Weeks	Carrier-side regulatory review of brand and campaign. Unregistered traffic is filtered or blocked outright. Cannot begin until the CPaaS vendor is chosen
Number porting	Days–weeks	Numbers can be unreachable during the porting window, and it is not quickly reversible. Any number printed on the website, in email signatures, on contracts or in Google Business is a live business dependency
WhatsApp template approval	Days	Meta reviews each message template before it can be sent outside a 24-hour session window

19.3 Inbound SMS and STOP handling

Covered by recipe §15.6. The STOP branch is a carrier requirement, not a feature. It ships with the first SMS send.

19.4 Other messaging channels

Channel	Replacement	Notes
WhatsApp	VENDOR	CPaaS WhatsApp Business API. Session-window rules apply — outside 24 hours only approved templates send
Facebook Messenger / Instagram DM	VENDOR	Meta APIs via n8n. **Verify usage.** A B2B compliance firm rarely closes deals in Instagram DMs
Google Business Messages	VENDOR	Google Business Profile API
Live chat widget	VENDOR	Chat vendor; n8n mirrors transcripts into `messageLog`
Slack	N8N	Native n8n node. No gap

20. Voice, IVR and telephony

20.1 What has to be replaced

GoHighLevel capability	CPaaS equivalent
Provisioned phone numbers	Numbers purchased or ported into the CPaaS account
Outbound calling / click-to-dial	CPaaS voice API plus a softphone or browser SDK
Inbound routing / IVR	Call-flow definition (TwiML or vendor equivalent), with n8n supplying dynamic decisions over HTTP
Call recording	CPaaS recording. **Storage, retention and consent notices become Aspire's responsibility**
Voicemail drop	CPaaS answering-machine detection plus pre-recorded audio
Missed-call text-back	Call-status webhook → n8n → SMS sub-workflow
Call reporting	`callLog` records in Twenty; reporting from Twenty views

20.2 IVR as a call flow

GoHighLevel's IVR actions — Gather Input on Call, Play Message, Connect to Call, End Call, Record Voicemail — are call-control primitives. They are rebuilt in the CPaaS, not in n8n. n8n's role is to answer questions the call flow asks:

```
Inbound call
  → CPaaS call flow: "Press 1 for support, 2 for sales"
  → Gather digit
  → CPaaS webhook → n8n:
        look up caller in Twenty by E.164 number
        return the routing target for that account's owner
  → CPaaS connects to the returned number
  → Call-status webhook → n8n → POST callLog
```

CALL RECORDING CARRIES OBLIGATIONS THE CRM MIGRATION DOES NOT

Recording consent requirements vary by jurisdiction, and Aspire operates from New York with clients in the US defence supply chain. Retention periods, storage location and access control for recordings are a compliance decision, not an engineering default. Confirm the current GoHighLevel settings during the manual sweep and carry the same or stricter rules forward — do not let them lapse silently at cutover.

21. Consent and suppression — the enforcement layer

THE HIGHEST-SEVERITY ITEM IN THE ENTIRE MIGRATION

Consent state currently lives in GoHighLevel and was **deliberately** made authoritative there. Removing GoHighLevel removes consent's home, and nothing else in the plan replaces it by default. If the account closes before the suppression list is exported and loaded, previously-unsubscribed people will be contacted again. For a company that sells compliance services, that is not an inconvenience — it is a demonstrable failure of the control Aspire advises clients to implement.

21.1 The design

`consentRecord` is per person, per channel, with a status, a source, a timestamp and a proof reference. Not a boolean on the contact — a boolean cannot answer "when, on what basis, and can we evidence it?", which is the only question that matters when someone complains.

Field	Purpose
`channel`	Email · SMS · Voice · WhatsApp · Postal · All
`status`	Opted In · Opted Out · Pending · Bounced · Complained
`source`	Form Submission · Manual · Imported from GHL · Unsubscribe Link · Reply STOP · Vendor Webhook
`effectiveAt`	When it took effect — the field that settles disputes
`proof`	Form submission id, IP, or reference to the evidence
`vendorReference`	ESP suppression-list entry id, for reconciliation

21.2 The three rules

1. **Populate before termination.** Export the GoHighLevel suppression list and DND flags, load into both `consentRecord` and the ESP suppression list. Verify counts match on both sides. This happens before any termination date is agreed, not before go-live.
2. **Every send checks it.** Enforced structurally by putting the check inside the `[VEND] Send *` sub-workflows (§14.4), so a workflow author cannot forget it — they never call the sending API directly. **A consent record that nothing consults provides no protection and false assurance, which is worse than none.**
3. **Vendor events write back.** ESP unsubscribes, hard bounces, spam complaints and SMS STOP replies all create `consentRecord` rows through n8n. Without this the record drifts within weeks and the CRM confidently reports consent that no longer exists.

21.3 Reconciliation

Two systems hold consent — Twenty as the auditable record, the ESP as the send-time enforcer. They will drift. A weekly reconciliation workflow is not optional:

```
[SYS] Consent Reconciliation — Schedule, weekly

1  GET all consentRecord where status = "Opted Out"
2  GET ESP suppression list via API
3  Code: diff the two sets
4  IF opted out in Twenty but absent from ESP  → add to ESP  (urgent)
5  IF suppressed in ESP but absent from Twenty → create consentRecord
6  Slack → #compliance: counts on both sides, and any discrepancy
7  POST automationRun
```

Direction 4 is the one that causes harm — someone who unsubscribed but is still reachable by the ESP. Alert on it separately rather than folding it into a summary.

Part V · Capture, scheduling and web

22. Forms and surveys

What GoHighLevel does. Hosted forms and surveys with a drag-and-drop builder, conditional logic, file upload, embedding on funnels or external sites, submissions landing directly on the contact record, and workflow triggering on submit.

The gap. Twenty has no public forms. Its `Form` workflow action collects input from a logged-in user during a manual workflow run — useful for internal data entry, but it is not a web form and cannot be exposed to a prospect.

22.1 Architecture

```
Prospect fills form (vendor-hosted or embedded)
   → Form vendor POSTs to n8n Webhook
   → n8n: verify signature · normalise · dedupe
   → Upsert Person + Company in Twenty
   → Create formSubmission record  (payload as RAW_JSON)
   → IF consent checkbox ticked → create consentRecord
   → Downstream: score, assign, task, acknowledge  (§15.1)
```

22.2 Vendor requirements

Requirement	Why
Webhook delivery with signature	The integration point. Unsigned webhooks are an open write endpoint into the CRM
Conditional logic	GoHighLevel forms commonly use it; check which of Aspire's actually do before paying for it
File upload	Relevant if any form collects documents
Spam protection	Otherwise n8n and Twenty absorb junk records
Embed on external site	Must sit on whatever hosts the web presence
Consent checkbox with proof	Feeds `consentRecord.proof`. Capture submission id and IP

22.3 Migration notes

- **Rebuild only forms with submissions.** The audit's per-form submission counts make this a data question, not a debate.
- **Every live form is a URL somewhere.** Embedded on the site, linked in an email signature, printed in a PDF, referenced in an ad. Inventory where each is referenced **before** switching, or leads silently stop arriving from a page nobody remembered.
- Keep the old form live and forwarding for a grace period after cutover where possible.
- Surveys and quizzes are the same architecture. If GoHighLevel surveys are unused — common — this is a DROP, not a vendor requirement.

23. Calendars and booking

What GoHighLevel does. Public booking pages, availability from connected calendars, buffers and minimum notice, round-robin and collective calendars, class and service booking, rooms and equipment, reminders, reschedule and cancel links, and appointment records on the contact.

The gap. Twenty has no booking engine, no availability calculation and no public booking surface. This is frequently the most-missed capability at cutover because it is customer-facing and breaks visibly.

23.1 Architecture

```
Prospect books via vendor booking page
   → Vendor webhook → n8n
   → Upsert Person
   → Create appointment record in Twenty
        { status:"Booked", appointmentType, scheduledAt,
          durationMinutes, meetingUrl, vendorEventId }
   → Twenty Record-created trigger → downstream logic
   → Reminders: scheduled n8n workflow  (§15.3)
   → Status changes: vendor webhook → PATCH appointment
   → No-show: scheduled query, past + still Booked  (§12.3)
```

23.2 Vendor requirements

Requirement	Why
Availability from connected calendars	Double-booking a client is a visible failure
Round-robin / team distribution	Only if GoHighLevel's is genuinely in use — check `calendar_groups`
Webhooks on book, reschedule, cancel	Without all three the Twenty record drifts from reality
Reschedule and cancel links	Reduces no-shows more than reminders do
API to generate single-use booking links	Replaces GoHighLevel's Generate One Time Booking Link action
Buffers, minimum notice, timezone handling	Timezone bugs in booking are unusually damaging to trust

> **BOOKING LINKS ARE PRINTED IN PLACES YOU DO NOT CONTROL**
>
> Existing GoHighLevel booking URLs appear in email signatures, on the website, in proposals, in calendar invitations already sent, and in ads. Inventory and redirect them. A dead booking link on a demo request page is a lead that never arrives and never complains.

24. Funnels, websites, blogs and tracked links

What GoHighLevel does. Funnel and website builder, hosting, custom domains, blogs, order forms and upsells, tracking codes, and page-level analytics.

The gap. Twenty has no page builder or hosting and never will — this is not a CRM function.

CHECK THIS BEFORE SCOPING ANY OF IT

Is aspiretss.com actually served from GoHighLevel, or only funnel subdomains? Aspire has operated since 2011 and almost certainly runs a real website on a real CMS. If GoHighLevel hosts only a handful of campaign funnels, this entire cluster shrinks from "replace a website platform" to "rebuild three landing pages on the existing CMS" — a difference of weeks. It is the single highest-leverage question in the manual sweep.

24.1 Replacement

Capability	Replacement	Notes
Landing pages / funnels	VENDOR	Existing CMS if there is one; otherwise a page builder. Registered in Twenty as `landingPage` records with visits and conversions
Websites	VENDOR	Verify first — likely already elsewhere
Blogs	VENDOR	CMS. n8n can publish programmatically, which supports the automated content pipeline in the wider automation plan
Blog authors / categories	VENDOR	Taxonomy moves with the CMS. Map author and category names during content migration — losing them breaks existing blog URLs and internal links
Order forms / upsells	VENDOR	Commerce surface — see §25
Custom domains / DNS	VENDOR	**Sequencing risk.** DNS changes propagate for hours and must be choreographed with the page migration
URL redirects	VENDOR	Move with hosting. Inventory every existing redirect — they exist because something linked to the old URL
Tracking codes / pixels	VENDOR	GA4, Meta, LinkedIn. Re-install on the replacement pages and verify events fire
Tracked / trigger links	N8N	**No vendor needed** — n8n serves the redirect (§18.4)

Part VI · Commerce

25. Products, quoting, invoicing, payments and contracts

What GoHighLevel does. Products and prices, invoices and recurring invoices, estimates and proposals, payment links, orders, coupons, subscriptions, documents and contracts with e-signature, and Stripe/PayPal/Authorize.net connections.

The gap. Twenty has no billing engine, no document generation and no payment processing. It records commercial state; it does not execute commerce.

25.1 The split

Capability	Replacement	Twenty object	Executed by
Products and prices	TWENTY	`product`	Native — catalogue only
Quotes / estimates (CPQ)	VENDOR	`quote`, `quoteLineItem`	Document generation + e-sign vendor
Invoices	VENDOR	`invoice`	Finance system
Payments / orders	VENDOR	`payment`	Processor
Subscriptions / recurring billing	VENDOR	`serviceSubscription`	Twenty **records** the subscription; the biller **charges** it
Coupons	VENDOR	—	Commerce surface
Documents and contracts	VENDOR	`quote.documentUrl`	E-sign vendor

25.2 Quoting — the CPQ gap

Twenty has no configure-price-quote capability. This maps to known gaps in the wider automation plan and is a genuine loss if quoting currently happens in GoHighLevel.

```
[BILL] Generate Quote — Manual trigger from Opportunity

1  Twenty manual trigger on an Opportunity
2  n8n: fetch opportunity + company + linked products
3  Code: calculate line totals, discounts, term value
4  POST quote + quoteLineItem records to Twenty
5  HTTP Request → document vendor: render from template
6  HTTP Request → e-sign vendor: send for signature
7  PATCH quote { status:"Sent", documentUrl }
8  Webhook on signature → PATCH quote { status:"Accepted" }
   → PATCH opportunity { stage:"Closed Won" }
   → POST serviceSubscription from the accepted line items
```

Step 8 is where the model earns its keep: an accepted quote becomes a subscription record automatically, which then drives the renewal escalation in §15.4. In GoHighLevel that chain does not exist because the subscription object does not exist.

25.3 Verify before building

> **FINANCE MAY ALREADY OWN ALL OF THIS**
>
> Aspire is a fourteen-year-old company selling subscription services to defence-sector clients. It has a finance function, and that function is unlikely to be invoicing from a marketing platform. **Confirm what actually issues Aspire's invoices before scoping cluster F.** If finance already uses an accounting system, the work here reduces to a read-only sync into Twenty so sales can see payment status — a fraction of the effort.

Part VII · Content, AI, reputation and community

26. AI features

GoHighLevel feature	Replacement	Build
Workflow AI (AI Prompt action)	N8N	Basic LLM Chain for single-prompt deterministic tasks — summarise, classify, draft. Direct replacement, with a better model than GoHighLevel's
Conversation AI (chat bot)	N8N + VENDOR	n8n AI Agent node with tools that read and write Twenty. Needs a channel to speak on — the bot is buildable, the channel is a vendor
Voice AI agents	VENDOR	Voice AI is a specialist capability. n8n orchestrates but does not synthesise or transcribe in real time. **Export the prompts regardless**
Content AI	N8N	LLM chain producing drafts into the CMS or an approval queue
Reviews AI	N8N	LLM drafts a review response; a human approves via the gate in §15.5
Workflow AI assistant (builder aid)	DROP	An authoring convenience, not a running feature. Nothing to migrate

26.1 Route every AI call through a gateway

Not per-workflow API keys. A single self-hosted gateway gives spend tracking per workflow, hard budget caps with automatic cut-off, one place to switch models, a request audit log, and redaction before data leaves the network.

> **TWO AI-SPECIFIC FAILURE MODES**
>
> **The agent self-trigger loop.** An agent triggered on record-update that then writes to the same record re-triggers itself, and unlike a sync loop this one costs money per iteration. Scope triggers to fields the agent never writes.
>
> **Failure by billing condition.** An agent call can fail because credits are exhausted, returning a failure result rather than a normal error. The error workflow must treat that as a distinct alertable condition, or the failure looks like an ordinary transient error and gets retried into a larger bill.

26.2 Data egress

Aspire's CRM holds prospect PII and client security-posture data. Before any of it reaches a third-party model, decide explicitly: which provider, whether data may leave the network at all, and what the gateway redacts. This is a governance decision with the same weight as the consent design, and it is currently unresolved.

27. Reputation, social and advertising

GoHighLevel feature	Replacement	Build

Review requests	VENDOR	Templated email or SMS with a review link, via the send sub-workflows. Trigger: `appointment.status` → Completed, or subscription milestone
Review monitoring	N8N	Scheduled polling of platform APIs → `review` records → Slack alert on anything below 4 stars
Review widgets on site	VENDOR	Moves with the web presence
Social planner	N8N or VENDOR	n8n plus platform APIs works but is fiddly to operate for a marketer. A dedicated scheduling tool is usually better value; either way `socialPost` records the result
Connected social accounts	VENDOR	Reconnection effort at cutover — OAuth per platform
Ad manager (Google / Facebook)	N8N	**Confirm whether ads are actually run from GoHighLevel** or natively in Ads Manager. Usually the latter, making this a DROP
Custom audiences / conversion API	N8N	Meta Marketing API and CAPI, GA4 Measurement Protocol, Google Ads offline conversions. All survive unchanged (§13.5)

28. Memberships, courses, affiliates and remaining modules

GoHighLevel feature	Replacement	Build
Courses / memberships	VENDOR / DROP	**Check aspireelearning.com first.** Aspire operates a dedicated security-awareness-training brand with its own portals. GoHighLevel memberships are far more likely a duplicate than a dependency
Client portal	VENDOR	Same question. If clients already log into an Aspire portal, this is DROP
Communities / groups	DROP	Frequently enabled, rarely used. Verify member counts before treating it as a requirement
Certificates	VENDOR	Relevant to A-SAT delivery; almost certainly already handled by the e-learning platform
Affiliate manager	VENDOR / DROP	No Twenty equivalent. **Does Aspire run an affiliate programme at all?** For a B2B compliance firm this is usually an unused module and the honest answer is DROP

> **THIS IS WHERE THE VENDOR COUNT FALLS**
>
> Clusters G and H exist in the catalogue because GoHighLevel ships these modules, not because Aspire uses them. Confirming that with evidence removes two of the eight vendor decisions outright and is a morning's work in the manual sweep. Do it early — it materially changes the number reported to management.

28.1 Remaining modules

The smaller GoHighLevel modules, listed so the coverage claim in this document is complete rather than nearly complete. Each is low-effort individually; collectively they are the kind of thing discovered on cutover day.

GoHighLevel feature	Replacement	Build and notes

Snippets / canned replies	TWENTY+N8N	Short reusable message text. Store as `emailTemplate` records with `purpose = Transactional`, or as `mergeVariable` entries for one-liners. If the inbox decision in §17 selects a shared-inbox vendor, that vendor has its own snippet feature and this becomes a DROP
Math / formula operations	N8N	n8n expressions or a Code node. Full JavaScript arithmetic rather than GoHighLevel's fixed operation list — this is an upgrade, not a workaround
Legacy campaigns (drip)	VENDOR / DROP	GoHighLevel's pre-workflow drip system. **Frequently forgotten and still running** — check `raw/campaigns.json` before assuming it is dead. Anything live rebuilds as a nurture sequence (§15.2); anything dormant is a DROP
Custom menu links	TWENTY	Sidebar links to external tools. Twenty supports workspace navigation customisation; low effort. Inventory what each link points at — some are shortcuts to systems nobody mentioned in the audit
Installed marketplace apps	N8N / DROP	**No public endpoint exposes these — enumerate by hand.** Each installed app is doing something to your data. Determine what, then rebuild in n8n or retire. A common source of "why did that stop working" after cutover
Native integrations Stripe · Google · Meta · QuickBooks · Shopify	N8N	Each needs its own reconnection plan and OAuth re-authorisation against the replacement stack. n8n has nodes for most. **QuickBooks and Stripe in particular may be finance-owned** — confirm the boundary before touching them
Inbound webhooks into GoHighLevel	N8N	External systems posting into GoHighLevel **break silently at cutover unless repointed** to n8n. On the pre-cutover checklist (§30.3) for exactly this reason
Zapier / Make connections	N8N	Often sit in a personal account and are invisible from inside GoHighLevel. **Ask teammates directly** — absence of evidence is not evidence of absence
SaaS mode / rebilling	DROP	Agency white-labelling — only relevant if Aspire resells GoHighLevel to clients. For internal use this is out of scope, but **confirm rather than assume**: if any client is being rebilled, terminating the account has contractual consequences well beyond this migration
Sub-account management	DROP	Twenty uses one workspace with a `brand` field separating Aspire TSS from Aspire eLearning (§7.2). If GoHighLevel runs multiple sub-accounts, each is a separate audit — confirm the count first, because it multiplies every estimate in this document

TWO OF THESE CARRY DISPROPORTIONATE RISK

Installed marketplace apps and **inbound webhooks into GoHighLevel** are the only items in this entire document that no API can enumerate for you. They must be listed by hand, and anything missed fails silently after the account closes — no error, no alert, just a process that quietly stopped. Both are in the manual sweep and on the pre-cutover checklist.

Part VIII · Reporting and delivery

29. Reporting, dashboards and attribution

GoHighLevel capability	Replacement	Build
Pipeline reporting	TWENTY	Twenty views grouped by stage, owner or service line. Direct equivalent for day-to-day sales reporting
Custom dashboards	TWENTY+N8N	Twenty dashboards are less mature than GoHighLevel's. **Confirm what is actually watched** — a dashboard nobody has opened in 90 days is a DROP, not a requirement
Call reporting	TWENTY+N8N	Views over `callLog`. Arguably better than GoHighLevel's, because it sits beside the account record
Appointment reporting	TWENTY+N8N	Views over `appointment`, including a no-show rate the scheduling vendor will not calculate
Email / campaign reporting	TWENTY+N8N	ESP webhooks update `campaign` aggregates; report from Twenty
Agent / user performance	TWENTY+N8N	Views grouped by owner across opportunities, tasks and `messageLog`
Attribution reporting	TWENTY+N8N	Source captured at ingest and carried on the Person. **Cross-system attribution needs a warehouse**, not Twenty views — that is a separate project, not part of this migration

> **ONE REPORT GOHIGHLEVEL COULD NEVER PRODUCE**
>
> `automationRun` makes automation health a CRM report: which workflows ran, how often, how many failed, and where. GoHighLevel does not expose execution history through its API at all — a limitation that materially obstructed this project's own audit. Building the write-back from day one means the next person to review this stack answers in one query what took weeks here.

30. Migration sequencing and cutover runbook

30.1 Phases

Phase	Work	Gate to exit	Runs in parallel with
0	**Decide vendors A and B**. Export the suppression list. Export AI prompts. Inventory phone numbers and live URLs	ESP and CPaaS contracted; suppression export verified complete	Nothing — this gates the schedule

1	Provision Twenty object model. Build the n8n foundation: error workflow, sub-workflow skeletons, credentials, naming	Foundation proven with one end-to-end test	Phase 0
2	Start email domain warm-up. A2P registration. Begin number porting	Warm-up underway; A2P submitted	Phases 3–4 — this is elapsed time, not effort
3	Migrate data: contacts, companies, opportunities, custom fields, tags. Load `consentRecord`	Record counts reconciled; consent counts match on both sides	Phase 2
4	Rebuild internal-only automations (§16.3 order 1–2)	Verified per §16.4	Phase 2
5	Stand up forms and scheduling vendors. Rebuild capture automations. Redirect live URLs	Test submission and booking flow end to end	Phase 2
6	Rebuild email automations. Low-volume real campaign	Deliverability verified with real recipients	—
7	Rebuild SMS and voice. Complete number porting	STOP handling proven; numbers answering	—
8	Parallel run. Reconcile daily	Exit criteria below	—
9	Cutover. Decommission GoHighLevel	—	—

30.2 Parallel-run exit criteria

A parallel run without measurable exit criteria drifts permanently into two systems of record with two sets of numbers that disagree. Set these before the window opens:

Criterion	Target
New records originating in Twenty	≥ 90%
Data parity — source records correctly mapped	≥ 99%
Reporting parity against current numbers	within 2% variance
Consent record count vs GoHighLevel suppression count	**exact match**
Automation failure rate over 7 days	< 1% of executions

30.3 Pre-cutover checklist

Every item verified, dated and initialled. Not asserted — verified.

Area	Checks
Consent	Suppression list exported · loaded into `consentRecord` · loaded into ESP · counts reconciled on both sides · every send path proven to check consent · STOP handling tested with a real number
Email	Domain warmed · DKIM/SPF/DMARC verified · bounce, complaint and unsubscribe webhooks live · templates rebuilt and rendered on real clients · real low-volume campaign delivered to inbox not spam
Telephony	A2P approved · numbers ported and answering · IVR flows tested · recording consent notices in place · call webhooks writing `callLog`
Capture	Every live form URL inventoried and redirected · every booking link inventoried and redirected · test submission and booking arrive in Twenty
Automation	Every rebuilt workflow verified per §16.4 · error workflow wired to all · alerting confirmed to a monitored channel · no workflow left referencing a GoHighLevel endpoint
Integrations	Every inbound webhook into GoHighLevel repointed · Zapier and Make connections migrated or retired · pixels and conversion APIs firing
Data	Record counts reconciled · custom fields populated · relations intact · backup taken and **restore tested**
People	Users trained · runbook written · named owner for each vendor relationship · someone other than the builder can operate it

> **DO NOT TERMINATE THE GOHIGHLEVEL ACCOUNT ON CUTOVER DAY**
>
> Keep it in read-only for a defined period — 30 to 60 days. The cost is trivial against the cost of discovering, in week three, that a form nobody inventoried was still feeding leads, or that a suppression entry was missed. **Termination is irreversible and the data is not recoverable.** Set the termination date separately from the cutover date, and only after a clean reconciliation.

31. Risk register

Risk	Severity	Impact	Mitigation
Consent data lost at termination	**Critical**	Unsubscribed people contacted again. Compliance breach at a company selling compliance	Export and reconcile before any termination date is agreed. §21
Cutover before domain warm-up	**Critical**	Campaigns land in spam; sender reputation damaged for months	Warm-up begins in Phase 2 and gates Phase 6
A2P not registered before SMS launch	High	Messages filtered or blocked by carriers	Submit in Phase 2; gate Phase 7

Number porting during business hours	High	Inbound calls unreachable	Schedule the porting window; publish a fallback number
Workflow missed in inventory	High	Automation dies silently; noticed weeks later	Manual inventory per §16; 365-day window on annual workflows; spot-check five
Live form or booking URL not redirected	High	Leads stop arriving from a page nobody remembered	URL inventory in the pre-cutover checklist
Sync or agent loop	High	API quota exhausted; records corrupted; unbounded LLM spend	Field-scoped triggers, `sourceSystem` stamps, execution ceilings. §14.3, §26.1
Twenty rate limit hit during migration	Medium	Migration stalls or partially completes	Batch 60, Loop Over Items, backoff on 429. §14.5
Hidden automation in personal Zapier/ Make	Medium	Breaks at cutover; owner may have left	Ask teammates directly — absence of evidence is not evidence of absence
Inbox loss rejected by users	Medium	Adoption failure regardless of technical success	Decide §17 from the channel histogram before cutover, not after complaints
Vendor selection delays everything	Medium	Schedule slips by the longest lead time	Phase 0 gates the plan explicitly
Single-person knowledge concentration	Medium	Nothing operable if one person is unavailable	Runbook, workflows in Git, named second owner
Email template rebuild underestimated	Low	Phase 6 overruns	`editorData` is not readable back — budget real rebuild time from the start

32. Cost model

Open source removes licence cost, not operational cost. Both belong in the number given to management.

> **Net position.** The only unavoidable recurring cost is one server plus the hours to operate it. Forms, scheduling and e-signature all turned out to have self-hosted answers, and transactional email fits inside the existing Workspace allowance at Aspire's volume. Telephony is the one genuinely priced cluster, and it is deferred because Aspire barely uses it.

Item	Basis	Note
Twenty licence	—	Self-hosted, no licence cost
n8n licence	—	Self-hosted community edition
Infrastructure	Per month	VPS for Twenty (4 GB+) and n8n, plus backup storage
Transactional email	Usually nil	The existing Google Workspace relay covers ~2,000 recipients/day. Above that, a relay such as Amazon SES at about $0.10 per 1,000. Take the volume from the audit, not an estimate
CPaaS	Per number + per message/minute	Plus A2P registration fees
Scheduling	Nil self-hosted	Cal.com runs on the same server. A hosted plan is a convenience choice, not a requirement
Forms	Nil	n8n hosts the form itself — no vendor at any volume. This was assumed to need one and does not
Commerce / e-sign	Nil self-hosted	DocuSeal runs on the same server and is eIDAS/ESIGN valid. Payments remain a finance decision
LLM usage	Per token	Measure during pilot; hard caps at the gateway
Operational load	Hours per month	Updates, backups, patching, vendor management, workflow maintenance. **Assign to a named owner**, do not absorb informally
Migration effort	One-off	Audit, rebuild, verification, parallel run

> **COMPARE LIKE WITH LIKE**
>
> The honest comparison is **one GoHighLevel subscription** against **infrastructure + four to six vendor subscriptions + operational hours**. The migration may still be correct — better data model, better automation, no per-seat CRM cost, no platform lock-in — but presenting only the removed licence cost is not a fair comparison and will not survive the first finance review.

33. What this plan does not solve

Stated plainly here so it is not discovered later.

- **The unified inbox does not come back.** Option B in §17 approximates it with another vendor. Options A and C do not. This is a real reduction in day-to-day convenience for reps and should be communicated before cutover, not defended after it.
- **Twenty's reporting is less mature** than GoHighLevel's dashboards. Adequate for pipeline and record reporting; weaker for marketing analytics. Cross-system attribution needs a warehouse, which is a separate project.
- **No native mobile app.** Twenty's web UI is responsive but not field-optimised. If anyone genuinely works from the GoHighLevel mobile app, this is a real gap with no clean answer.

- **Performance at Aspire's real data volume is unproven.** Local Docker testing cannot establish it, and it remains the largest untested assumption. Two items previously listed here have since been settled: backup and restore is now verified by a script that dumps each database, restores it into a scratch database and compares row counts exactly, recording the date; and TLS with a reverse proxy has been stood up and exercised — certificates issued, traffic proxied, HTTP redirected, security headers applied.
- **More systems means more failure modes.** One vendor relationship becomes four to six, each with its own outages, billing and support. The operational surface grows even as the licence cost falls.
- **Deliverability becomes Aspire's responsibility.** GoHighLevel managed sending reputation invisibly. Someone must now own it by name.
- **This document assumes the audit's findings.** It describes how to replace capabilities. It does not know which ones Aspire uses. Building from it without the audit would replace things nobody needs — and the audit typically removes a third to a half of the surface.

> **THE FAIR SUMMARY**
>
> Twenty and n8n replace GoHighLevel's CRM and automation completely, and improve on both. **The communication layer proved cheaper to replace than this document originally assumed.** Public forms are hosted by n8n itself; booking and e-signature have credible self-hosted answers in Cal.com and DocuSeal; transactional email goes through the existing Google Workspace relay, with a paid relay needed only above roughly 2,000 recipients a day. SMS, voice and WhatsApp genuinely require a licensed carrier — but Aspire barely uses them, so they are a deferred decision rather than a blocking dependency. What remains is therefore an unbundling exercise with no irreducible procurement dependency, and a schedule set by engineering effort and data migration rather than by vendor lead times. The remaining hard constraints are operational, not commercial: someone must own deliverability by name, and the system needs a second person who can operate it.

Appendices

Appendix A · Complete feature disposition matrix

All 111 catalogued GoHighLevel features across 23 areas, with disposition and the section of this guide that specifies the replacement. Generated directly from `reference/ghl_feature_taxonomy.csv`, so it cannot drift from the audit instrument.

TWENTY Native **N8N** n8n alone **TWENTY+N8N** Record + logic **VENDOR** Third party required **DROP?** Verify usage, likely retire

Data Model — see §7

ID	Feature	Disposition	Notes
DM-01	Contact custom fields	TWENTY	Map to Person/Company fields. Count fields with <2% fill as DROP candidates.
DM-02	Opportunity custom fields	TWENTY	Map to Opportunity fields.
DM-03	Custom objects	TWENTY	Twenty custom objects are stronger than GHL's. Direct win.
DM-04	Custom values (merge variables)	TWENTY+N8N	Twenty has no account-level variable store. Hold in n8n env or a settings object.
DM-05	Tags	TWENTY	Check whether tags encode workflow state rather than describing contacts — those become fields, not tags.
DM-06	Companies / businesses	TWENTY	Maps to Twenty Company.
DM-07	DND / consent flags	VENDOR	Consent must be enforced at send time. Twenty cannot enforce it because Twenty does not send campaigns.

Contacts & Segmentation — see §9

ID	Feature	Disposition	Notes
CT-01	Smart lists / saved filters	TWENTY	Twenty saved views are equivalent.
CT-02	Bulk actions on contacts	TWENTY+N8N	Twenty supports bulk edit; bulk workflow enrolment differs.
CT-03	Import / export CSV	TWENTY	Twenty has native CSV import.
CT-04	Duplicate detection / merge	TWENTY+N8N	Dedup logic belongs in n8n at ingest.
CT-05	Manual actions queue (call/SMS tasks)	TWENTY+N8N	Twenty Tasks cover the queue; the dialer does not exist.
CT-06	Notes and tasks on records	TWENTY	Native.

Opportunities & Pipelines — see §8

ID	Feature	Disposition	Notes
OP-01	Pipelines and stages	TWENTY	Direct map. Count stages per pipeline.
OP-02	Opportunity value / forecasting	TWENTY+N8N	Twenty forecasting is weaker than GHL. Confirm what is actually used before calling it a gap.
OP-03	Pipeline automation triggers	TWENTY	Twenty record-updated triggers cover stage changes.

Conversations & Inbox — see §17, §19

ID	Feature	Disposition	Notes
CV-01	Unified inbox	VENDOR	Twenty has no inbox. Email threads sync to records only.
CV-02	SMS conversations	VENDOR	No SMS in Twenty.
CV-03	WhatsApp	VENDOR	n8n can route via a provider; needs vendor decision. Confirm the type enum values against the first real pull — an unrecognised field yields unknown, not no, so this fails…
CV-04	Facebook / Instagram DM	VENDOR	Vendor decision.
CV-05	Google Business Messages	VENDOR	Vendor decision.
CV-06	Live chat widget	VENDOR	No equivalent.
CV-07	Snippets / canned replies	VENDOR	Tied to the inbox that does not exist.

Email & Templates — see §18

ID	Feature	Disposition	Notes
EM-01	Email templates	TWENTY+N8N	Twenty Send Email has no template library. Templates live in n8n or the sending vendor.
EM-02	Drag-and-drop email builder	VENDOR	editorData is not readable back via API — HTML export only. Rebuild cost is real.
EM-03	Bulk / campaign email send	VENDOR	HARD CEILING. Twenty Send Email = synced mailbox, one recipient per action.
EM-04	Drip sequences	TWENTY+N8N	n8n Wait + Twenty Delay can sequence; sending still needs a vendor.
EM-05	Trigger links / click tracking	N8N	n8n webhook redirect can reproduce this.
EM-06	Dedicated sending domain / DKIM / SPF	VENDOR	Deliverability infrastructure. Vendor decision.
EM-07	Unsubscribe / suppression list	VENDOR	Compliance-critical. Must not be lost at cutover.
EM-08	Email verification	N8N	n8n plus a verification API.

EM-09	Scheduled email sends	TWENTY+N8N	Twenty schedule trigger plus n8n.

SMS & Telephony — see §19, §20

ID	Feature	Disposition	Notes
SM-01	SMS sending	VENDOR	HARD CEILING. Vendor decision.
SM-02	Phone numbers (LC Phone)	VENDOR	Number ownership and porting. Vendor decision.
SM-03	A2P 10DLC registration	VENDOR	Regulatory registration is tied to the number provider. Long lead time — flag early.
SM-04	Inbound call routing / IVR	VENDOR	Vendor decision.
SM-05	Call recording	VENDOR	Retention and consent implications.
SM-06	Voicemail drop	VENDOR	Vendor decision.
SM-07	Missed-call text-back	VENDOR	Depends on telephony vendor.

Voice & AI Features — see §26

ID	Feature	Disposition	Notes
AI-01	Voice AI agents	VENDOR	Export the prompts regardless of disposition — the prompt is the asset.
AI-02	Conversation AI bot	N8N	n8n plus LLM gateway can reproduce, if a channel exists to speak on.
AI-03	Content AI	N8N	n8n plus LLM gateway.
AI-04	Reviews AI	N8N	Depends on review platform access.
AI-05	Workflow AI assistant	DROP?	Authoring aid, not a running feature. Usually safe to drop.

Calendars & Scheduling — see §23

ID	Feature	Disposition	Notes
CL-01	Calendars and availability	VENDOR	Twenty has no booking engine.
CL-02	Public booking pages	VENDOR	Vendor decision — this is usually the most-missed feature at cutover.
CL-03	Round-robin / team distribution	VENDOR	Assignment logic can move to n8n; the booking surface cannot.
CL-04	Appointment reminders	TWENTY+N8N	Sending channel still needs a vendor.
CL-05	Booking volume (evidence)	TBC	Evidence row. Zero bookings in 90 days makes CL-01..04 DROP candidates.

Forms & Surveys — see §22

ID	Feature	Disposition	Notes
FS-01	Forms	VENDOR	Twenty has no public forms. n8n webhook plus a hosted form tool.
FS-02	Form conditional logic	VENDOR	Depends on replacement form tool.
FS-03	Surveys / quizzes	VENDOR	Same as FS-01.
FS-04	Form submission volume (evidence)	TBC	Evidence row. Drives DROP decisions across FS-01..03.
FS-05	Survey submission volume (evidence)	TBC	Evidence row.

Funnels & Websites — see §24

ID	Feature	Disposition	Notes
FW-01	Funnels	VENDOR	No equivalent. Confirm whether marketing owns these outside GHL already.
FW-02	Websites / pages	VENDOR	Check whether aspiretss.com is actually hosted here or on WordPress.
FW-03	Order forms / upsells	VENDOR	Commerce surface.
FW-04	Custom domains	VENDOR	DNS cutover — sequencing risk at go-live.
FW-05	Tracking codes / pixels	VENDOR	Move with whatever hosts the pages.

Blogs & Content — see §24

ID	Feature	Disposition	Notes
BL-01	Blog sites	VENDOR	Relevant to S-02 blog pipeline in the 82-item plan.
BL-02	Blog posts	VENDOR	n8n can publish to a replacement CMS.
BL-03	Blog authors / categories	VENDOR	Moves with the CMS.

Workflows & Automation — see §11-16

ID	Feature	Disposition	Notes
WF-01	Workflow inventory	TWENTY+N8N	Metadata only from API. Count published vs draft.
WF-02	Workflow internals (steps and actions)	TWENTY+N8N	NOT available on any public endpoint. Requires the authenticated-session route.
WF-03	Workflow triggers	TWENTY+N8N	Map each GHL trigger to a Twenty trigger or an n8n trigger.
WF-04	If/else and conditional branching	N8N	n8n Switch/IF. Twenty Filter is weaker.
WF-05	Wait / delay steps	TWENTY+N8N	Both have this.

WF-06	Goal / exit conditions	N8N	No native equivalent. Rebuild as explicit checks.
WF-07	Math / formula operations	N8N	n8n Code node.
WF-08	Outbound webhooks from workflows	TWENTY+N8N	Twenty HTTP Request action.
WF-09	Custom code steps	TWENTY+N8N	Twenty Code action plus n8n Code node.
WF-10	Legacy campaigns (drip)	VENDOR	Often forgotten and still running. Check before assuming dead.
WF-11	Workflow execution history / enrolment counts	TBC	NOT exposed by the public API. This is the DROP evidence gap — see docs/02.

Payments & Commerce — see §25

ID	Feature	Disposition	Notes
PY-01	Products and prices	TWENTY+N8N	Model as Twenty records; billing stays external.
PY-02	Invoices	VENDOR	Twenty has no invoicing.
PY-03	Estimates / quotes (CPQ)	VENDOR	Known Twenty gap. Maps to SA-04/SA-05 in the 82-item plan.
PY-04	Subscriptions / recurring billing	VENDOR	Twenty can RECORD subscriptions (Service Subscription object) but cannot BILL them.
PY-05	Payment processor connections	VENDOR	Stripe/PayPal connection moves with the billing surface.
PY-06	Coupons	VENDOR	Commerce surface.
PY-07	Documents and contracts (e-sign)	VENDOR	Vendor decision. Check whether legal already uses something else.

Memberships & Courses — see §28

ID	Feature	Disposition	Notes
MC-01	Courses / memberships	VENDOR	Check overlap with aspireelearning.com — may already live there.
MC-02	Client portal	VENDOR	Vendor decision.
MC-03	Communities	DROP?	Frequently enabled and unused. Verify before escalating.
MC-04	Certificates	VENDOR	Relevant to A-SAT training delivery.

Reputation & Reviews — see §27

ID	Feature	Disposition	Notes
RV-01	Review requests	VENDOR	Requires a sending channel.
RV-02	Review monitoring / widgets	N8N	n8n plus platform APIs.

Social & Ads — see §27

ID	Feature	Disposition	Notes
SO-01	Social planner scheduling	N8N	n8n plus platform APIs, or a dedicated tool.
SO-02	Connected social accounts	VENDOR	Reconnection effort at cutover.
SO-03	Ad manager (Google / Facebook)	N8N	Confirm whether ads are actually run from GHL or natively.

Reporting & Dashboards — see §29

ID	Feature	Disposition	Notes
RP-01	Attribution reporting	TWENTY+N8N	Cross-system attribution needs the AN-01 warehouse, not Twenty alone.
RP-02	Call reporting	VENDOR	Moves with telephony.
RP-03	Appointment reporting	VENDOR	Moves with scheduling.
RP-04	Agent / user performance reporting	TWENTY+N8N	Twenty views cover some of this.
RP-05	Custom dashboards	TWENTY+N8N	Twenty dashboards are less mature. Confirm what is actually watched.

Users & Permissions — see §10

ID	Feature	Disposition	Notes
US-01	User accounts and seats	TWENTY	Map to workspace members. Count active seats.
US-02	Roles and permission levels	TWENTY	Twenty object-level roles. Field-level restriction is thinner.
US-03	Teams	TWENTY	Confirm whether teams drive routing logic.
US-04	SSO	VENDOR	Twenty SSO is Organization-tier. Confirm whether Aspire requires it.

Settings & Business Profile — see §6, §24

ID	Feature	Disposition	Notes
ST-01	Business profile / hours	TWENTY	Low effort.
ST-02	Custom menu links	TWENTY	Minor.
ST-03	URL redirects	VENDOR	Moves with hosting.
ST-04	Domains and DNS	VENDOR	Sequencing risk at go-live.

Integrations & Marketplace — see §30

ID	Feature	Disposition	Notes

IN-01	Installed marketplace apps	TBC	No public read endpoint. Must be enumerated by hand.
IN-02	Native integrations (Stripe/ Google/Meta/QuickBooks)	N8N	Each needs its own reconnection plan.
IN-03	Zapier / Make connections	N8N	Hidden automation often lives here. Check explicitly.
IN-04	Inbound webhooks into GHL	N8N	External systems posting into GHL will break at cutover unless repointed.

Agency & Multi-Location — see §30

ID	Feature	Disposition	Notes
AG-01	Sub-account count	TBC	Determines audit scope. Answer before estimating effort.
AG-02	Snapshots	DROP?	Audit instrument rather than a feature to migrate.
AG-03	SaaS mode / rebilling	TBC	Only relevant if Aspire resells GHL to clients. Confirm.

Media & Assets — see §7

ID	Feature	Disposition	Notes
MD-01	Media library	TWENTY+N8N	Twenty attachments cover records; hosted assets for pages do not move.

Mobile & Field Access — see §33

ID	Feature	Disposition	Notes
MO-01	Mobile app	VENDOR	Twenty has no native mobile app. Confirm whether anyone actually uses the GHL app.

Totals

Disposition	Count	Meaning
TWENTY	15	Native Twenty. Configuration only.
N8N	14	n8n reproduces it with no Twenty involvement.
TWENTY+N8N	20	Record in Twenty, logic in n8n.
VENDOR	52	Third-party service required — consolidates into 8 decisions (§5).
DROP?	3	Likely unused or duplicated elsewhere. Verify, then retire.
TBC	7	Evidence row — resolved by the audit, not by this guide.

> **THIS IS A PRE-AUDIT UPPER BOUND**
>
> Every feature GoHighLevel offers is counted, not every feature Aspire uses. The usage audit typically removes a third to a half of the surface, and four of the eight vendor clusters may collapse entirely once measured. Report this table as a ceiling, never as a workload.

Appendix A · How this fails silently

Everything in the preceding sections describes how the replacement is designed. This appendix describes how it breaks, and it exists because the gap between the two turned out to be the whole risk in this project.

Every item below was found by building the system and running it, not by reading documentation. Each one produced a workflow that looked correct in the editor, deployed without error, and did the wrong thing — in most cases with no error logged anywhere. Two of them sent real output to the wrong recipient.

The pattern worth internalising. In this stack, a malformed instruction is usually not rejected. It is ignored, and the surrounding logic carries on with whatever it got. A migration is therefore not validated by "it deployed" or "no errors in the log" — only by asserting on the records that were supposed to be created.

A.1 Twenty silently ignores a malformed query

This is the most dangerous single behaviour in the platform, and the one that justifies the validator in §A. 6.

Written as	What actually happens
`?filter[email][eq]=x`	**The filter is discarded and the entire table is returned.** HTTP 200. No warning
`?orderBy=field[DescNullsLast]`	Sort discarded, arbitrary order returned. HTTP 200
`?filter=status[in]:A,B`	400 — `in` requires a bracketed array, `[A,B]`

The correct forms are `?filter=field[eq]:value` and `?order_by=field[DescNullsLast]`. Comma separates conditions and means AND.

In this build, the lead-capture workflow used the bracketed form to find an existing contact by email. It matched nothing, so it received every person in the database and used the first row. The consequence was an acknowledgement email addressed to an unrelated contact, and a consent check that read a different person's consent record. The only visible symptom was a wrong first name in an email nobody was reading closely.

Every interpolated filter value must be URL-encoded. A space, bracket or colon in the value corrupts the query into a 400. The error handler's own deduplication query broke this way on a workflow name containing brackets: the failure was recorded, the handler then errored on the next node, and the alert was never sent. Values reaching these filters include user-typed email addresses and a slug taken from a public URL, so this is not only about awkward internal names.

A.2 n8n expressions end at the first `}}`

An expression containing a nested object literal terminates early, and the remainder is parsed as code:

```
{{ JSON.stringify({"phones": {"primaryPhoneNumber": $json.phone}}) }}
                                                    ^^ ends here
```

Twenty's composite fields — `name`, `emails`, `phones`, `domainName`, `bodyV2` — are all nested, so this affects most write operations. Emit `} }` with a space. The generator in this build lints for it and refuses to write a workflow containing one.

Relatedly, `jsonBody` is only evaluated when the **whole** parameter begins with `=`. Marking the inner values instead sends the body as literal text: the API receives `{"clickCount": "={{ $json.count }}"}` and rejects it, or worse, accepts the string where a number was meant.

A.3 References resolve by id, not by name

Sub-workflow calls and `settings.errorWorkflow` both look like they accept a name. They do not — they resolve by internal id. Given a name, they dangle silently.

The effect is that **every workflow appears to have error handling and none of it fires**. Nothing warns; the error workflow simply is not invoked. In this build the deployment script binds ids at deploy time, which is why the workflow library in version control can still reference them by name and stay portable between instances.

A.4 Error workflows do not fire for manual runs

`settings.errorWorkflow` is invoked only for **production** executions. A workflow run by hand in the editor fails in the editor, and the error workflow is never called.

This means testing failure handling by clicking "Execute Workflow" proves nothing whatsoever. The only way to verify it is to trigger a real failure through a real trigger — which is why this build ships a deliberately-failing webhook-triggered workflow used solely to exercise the alerting path.

Similarly, **a webhook responds on receipt, before the workflow runs**. A 200 from a webhook says nothing about the outcome. Assert on the record that should have been written, never on the HTTP status.

A.5 Version-specific traps

Trap	Consequence
Form inputs post as `field-0`, `field-1` … by **position**, not by label	Only affects scripted submissions; a browser gets it right. A script posting by label receives 200 with every value silently null
Form path moved from `path` to `options.path` at typeVersion 2.2	Set only one and the other version falls back to a random UUID — the form works, at a URL no campaign can link to. Set both
Task body is `bodyV2`, a rich-text composite taking `{"markdown": "..."}`	A bare string is a 400; a plain `body` key is not a field at all
Code nodes run in a sandboxed task runner	`process` does not exist. Use `$env`
A Switch's `fallbackOutput` must be an existing index or `"extra"`	Naming a non-existent output throws at runtime, not at save time
Default opportunity stages are `NEW, SCREENING, MEETING, PROPOSAL, CUSTOMER`	There is no `WON`. Filtering on it is a 400
`/rest/metadata/*` rejects every query string, and returns only a slice of each object's fields	It is not a complete schema. Merge it with the keys of a real record. Reading it alone makes existing fields look absent
Twenty's rate limit is 100 requests per **minute**	Retries three times two seconds apart give up six seconds into a sixty-second window. Use the longest backoff the platform allows

A.6 What this implies for the migration method

Because failures are silent, correctness has to be asserted rather than assumed. Three gates were built for this migration and each one caught defects the others did not:

Gate	What it catches
Generation-time lint	Refuses to emit a workflow containing an expression that would fail at runtime — truncated expressions, un-encoded filter values, bodies that are not marked as expressions
Schema validation	Checks every API call against the live instance: object exists, field exists, enum value is declared, filter and sort syntax correct. Reconstructing the schema requires merging the metadata endpoint with a real record, because neither is complete alone
Behavioural proof	Submits the real form, follows the real redirect, breaks something on purpose, triggers the real schedules — then asks the CRM whether the records exist. This is the only gate that can catch a silently-ignored filter

The third is the one that matters. A workflow can pass every static check and still write to the wrong record, and no amount of code review finds it. The question a migration must be able to answer is not "did it run" but "is the right row in the database now".

All defects in this appendix were found and fixed during the build. They are recorded here because the same platforms will behave the same way for the next person, and because a document that describes only the happy path is the reason projects like this are declared finished three weeks early.

Appendix B · n8n node reference for this build

The nodes this migration actually uses. Not the full catalogue — the working set.

Core flow

Node	Use in this build
Webhook	Entry point for every vendor and Twenty event. Captures method, headers, query and body
Respond to Webhook	Controls the response. Required for the tracked-link 302 redirect and for fast webhook acknowledgement
Schedule Trigger	Every "absence or elapsed time" pattern: stale, overdue, no-show, birthday, renewal, reminder
Error Trigger	First node of the workspace error workflow. Fires after a workflow fails
HTTP Request	All Twenty Core and Metadata API calls, and any vendor without a dedicated node
Code	HMAC verification, scoring, deterministic round-robin, payload shaping. JavaScript or Python
IF / **Switch**	Two-branch and multi-branch logic. Replaces GoHighLevel If/Else
Filter	Drop items that should not continue — cheaper than an IF with an empty branch
Merge	Recombine branches; join data from two sources
Wait	Duration, specific time, or **On Webhook Call** using `$execution.resumeUrl` — the approval gate
Loop Over Items	Batching for Twenty's 100-per-minute limit; replaces GoHighLevel Drip Mode
Split Out / **Aggregate**	Array handling; replaces GoHighLevel Arrays
Set / **Edit Fields**	Normalisation — lowercase email, E.164 phone, trim
Execute Sub-workflow	Calls `[VEND] Send *`. The structural backbone of §14.4
Stop and Error	Explicit abort that reaches the error workflow. Replaces Remove from Workflow
NoOp	Clean branch termination

Application nodes

Node	Use
Slack	Internal notifications, alerts, approval prompts. Direct replacement for GoHighLevel's Slack action
Twilio	SMS, MMS and WhatsApp — the WhatsApp toggle routes through the Business API
Google Sheets	Direct replacement for GoHighLevel's Sheets action
Facebook Lead Ads	Replaces the Facebook Lead Form Submitted trigger
Shopify	Abandoned checkout, order placed, order fulfilled — if Shopify is in use at all
Basic LLM Chain	Single-prompt deterministic AI: summarise, classify, draft. Replaces Workflow AI
AI Agent	Tool-using agents. Reserve for tasks that must decide at runtime — it costs more per run

Node settings that matter

Setting	Where and why
Retry On Fail	Every node touching an external API. 2–3 attempts with a wait handles transient failures and 429s
On Error	Stop Workflow (default) · Continue · Continue using error output. Use Continue only for genuinely non-critical steps
Error Workflow	Workflow Settings. Set on **every** production workflow
Always Output Data	Off by default; turning it on hides empty results and masks real failures
Execution order	Set explicitly on workflows with parallel branches where ordering matters

Appendix C · Twenty API reference

Endpoints and authentication

Item	Value
Core API	`/rest/` · `/graphql/` — CRUD on all objects
Metadata API	`/rest/metadata/` · `/metadata/` — objects, fields, relations
Base URL (self-hosted)	`https://{your-domain}/` — must match `SERVER_URL`
Authentication	`Authorization: Bearer <API_KEY>`
Key creation	Settings → API & Webhooks → Create key
Key scoping	Settings → Members → Roles → Assignment
Playground	Settings → API & Webhooks, once a key exists
Rate limit	**100 requests per minute**
Batch size	**60 records per call**; GraphQL also supports batch upsert
Schema	Per workspace — a new custom object immediately gets endpoints under its own name

Query syntax — get this wrong and it fails silently

Twenty does not reject a malformed filter. Send the bracketed shape most people try first and it **ignores the filter and returns the whole table**. No error, no log entry — the workflow simply reads the first row of every record. In our own build this sent a lead acknowledgement to an unrelated contact, and the only symptom was the wrong first name in the email. `orderBy=` is ignored the same way.

Purpose	Correct	Wrong, and what happens
Filter	`?filter=field[eq]:value`	`?filter[field][eq]=value` — returns everything
Multiple	`?filter=a[eq]:1,b[eq]:2` (comma = AND)	—
In a set	`?filter=status[in]:[A,B]`	bare comma list — 400
Substring	`?filter=domainName.primaryLinkUrl[ilike]:%acme%`	—
Sort	`?order_by=field[DescNullsLast]`	`orderBy=` — silently ignored
Relation	`?filter=personId[eq]:<uuid>`	the relation is `person`; the filter key is `personId`

`/rest/metadata/*` rejects every query string, and `/rest/metadata/objects` embeds only a slice of each object's fields — so it is not a complete schema. Merge it with the keys of a real row from `/rest/{object}?limit=1`.

Common calls

```
# Find a person by email  (idempotency check before create)
GET /rest/people?filter=emails.primaryEmail[eq]:someone@example.com

# Create
POST /rest/people
{ "name": {"firstName":"A","lastName":"B"},
  "emails": {"primaryEmail":"a@b.com"},
  "ghlContactId": "abc123" }

# Update
PATCH /rest/people/<id>
{ "leadScore": 72 }

# Custom object — endpoint follows namePlural
POST /rest/serviceSubscriptions
{ "serviceLine":"SOCAAS", "status":"ACTIVE",
  "renewalDate":"2027-03-01", "mrr":{"amountMicros":450000000,
  "currencyCode":"USD"}, "companyId":"<uuid>" }

# Create a field
POST /rest/metadata/fields
{ "objectMetadataId":"<uuid>", "name":"renewalDate",
  "label":"Renewal Date", "type":"DATE", "icon":"IconCalendar" }

# Create an object
POST /rest/metadata/objects
{ "nameSingular":"consentRecord", "namePlural":"consentRecords",
  "labelSingular":"Consent Record", "labelPlural":"Consent Records",
  "icon":"IconShieldLock" }
```

Webhooks

Property	Value
Direction	Twenty **sends** only. There is no inbound webhook handler on objects — all writes go through the Core API
Events	Record created, updated, deleted — e.g. `person.created`
Payload	`{ event, data, timestamp }`
Signature	HMAC SHA256 over `${timestamp}:${payload}`
Headers	`x-twenty-webhook-timestamp` · `x-twenty-webhook-signature`
Filtering	**None at source.** All event types are delivered — filter early in n8n and exit cheaply

Field types

`TEXT` · `NUMBER` · `BOOLEAN` · `DATE` · `DATE_TIME` · `CURRENCY` · `SELECT` · `MULTI_SELECT` · `RELATION` · `LINKS` · `EMAILS` · `PHONES` · `ADDRESS` · `RATING` · `RAW_JSON` · `ARRAY` · long text · domain

SELECT and MULTI_SELECT options require `label`, `value`, `position` and `color`. Relation payload shapes vary between Twenty versions — verify against the deployed instance rather than assuming.

Appendix D · Sources

Claims about GoHighLevel and Twenty capabilities in this document were verified against these sources during preparation, August 2026. Where a claim could not be verified, the text says so.

GoHighLevel

- A List of Workflow Actions — HighLevel Support Portal (complete action inventory used in §13)
- List of Workflow Triggers in HighLevel (trigger inventory used in §12)
- Get Workflow — HighLevel API documentation (confirms metadata-only response)
- API endpoints: POST/PUT/DEL for workflows — HighLevel Ideas ("The platform API is read-only for workflows… There are no write endpoints available." Open since 2022)
- Get Custom Fields by Object Key — HighLevel API ("Only supports Custom Objects and Company (Business) today.")
- Private Integration Tokens — HighLevel API (auth, headers, rotation)
- API Versioning — HighLevel API (date-based versions and v3)
- Snapshots Overview — HighLevel Support Portal
- HighLevel 2026 feature updates (website visit, invoice, no-show and subscription triggers)

Twenty

- Workflow Actions — Twenty Documentation ("only one recipient is possible at the moment")
- Workflow Triggers — Twenty Documentation
- Fields — Twenty Documentation (18 field types)
- APIs — Twenty Documentation (Core and Metadata APIs, 100 req/min, 60-record batches)
- What is Twenty — Twenty Documentation
- Twenty webhook HMAC signature format and headers

n8n

- Core nodes — n8n Documentation
- Webhook node, Respond to Webhook node, Wait node (`$execution.resumeUrl`)
- Error Trigger node and error-handling guidance (Retry On Fail, On Error options)
- Twilio node (SMS, MMS, WhatsApp toggle)

Internal

- Aspire CRM Project — Master Context, 8 August 2026
- `reference/ghl_feature_taxonomy.csv` — 111-feature catalogue
- `reference/twenty_schema.yaml` — 25-object parity model
- `docs/02-ghl-api-findings.md` · `docs/03-replacement-stack.md` · `docs/06-twenty-object-model.md`

> **VERIFICATION STANDARD**
>
> Product capabilities change. Every capability claim here should be re-verified against the deployed Twenty instance and the live vendor APIs before being relied upon for a build decision — particularly Twenty's relation-creation payload shape and any GoHighLevel endpoint behaviour, which the audit's discovery run measures directly rather than assuming.